

Overview

Slides modified from Angel book and Scott King

Overview

- Geometry Planner Projection

$$\text{Projection} * \text{View} * \text{Model} * P$$

- Shading

$$K_d I_d (n \cdot L) + K_s I_s (v \cdot r)^\alpha + K_a I_a$$

Overview

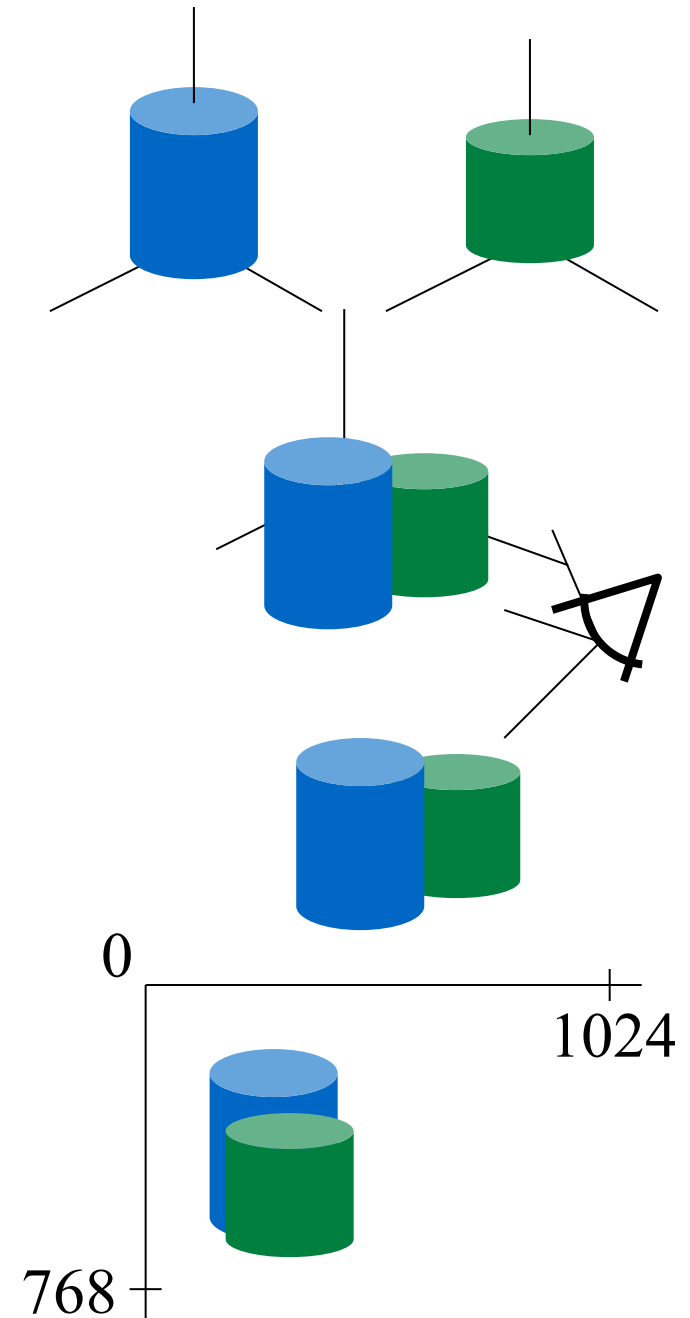
- Geometry Planner Projection

Projection * View * Model * P

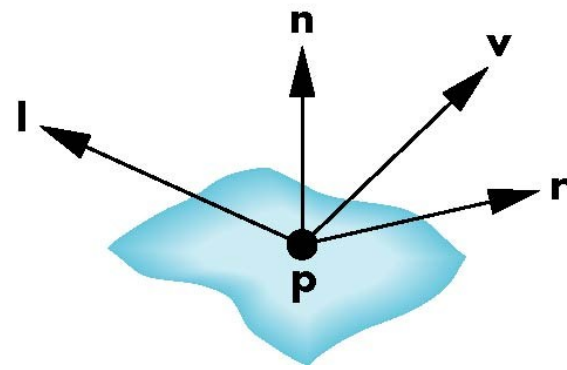
- P is a point of an object
- The purpose of the matrix is to map 3D geometry of the scene to an image on the screen.
- Projection, View, Model are sequence of 4x4 matrices the breaks down this mapping into manageable independent subset.
- Model matrix places an object into a world
 - Each object could have its own model matrix to manipulate its transformation
- View matrix places at the origin of the scene
 - It is shared between all objects, and every object is measured with respect to the camera
- Projection has two purposes placing a lens to the camera and map the scene to the gl native viewing volume
- ***All the matrices are invertible**

Overview

- **Object Coordinate System**
 - From the objects point of view
- **World Coordinate System**
 - For the entire world/scene
- **Camera Coordinate System**
 - From the cameras point of view
- **Screen Coordinate System**
 - In the screen (or window)
- Often refer to as “space”



Overview



- Shading

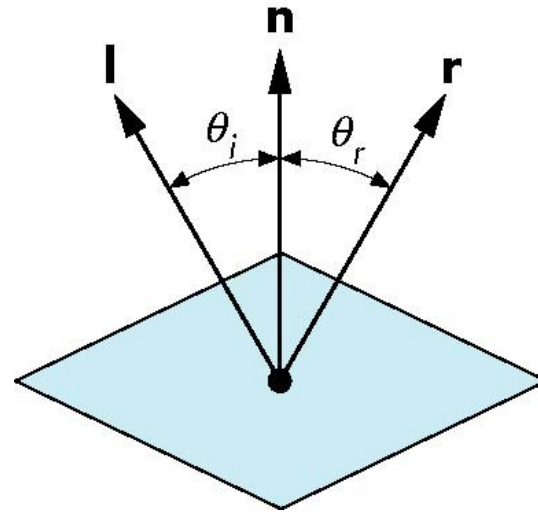
$$K_d I_d (\mathbf{n} \cdot \mathbf{L}) + K_s I_s (\mathbf{v} \cdot \mathbf{r})^\alpha + K_a I_a$$

- Phong illumination model is a rough approximation for material light interaction
- I terms define light intensities
- K terms define material properties
- The value of the term determine the color at point P
- Vectors n, l, v, and r are calculated based on light position, camera, point, the normal requires knowing the triangle
- All these vectors need to be unit vectors

Overview

- Normal is determined by local orientation
- Angle of incidence = angle of reflection

$$\mathbf{r} = 2 (\mathbf{l} \cdot \mathbf{n}) \mathbf{n} - \mathbf{l}$$

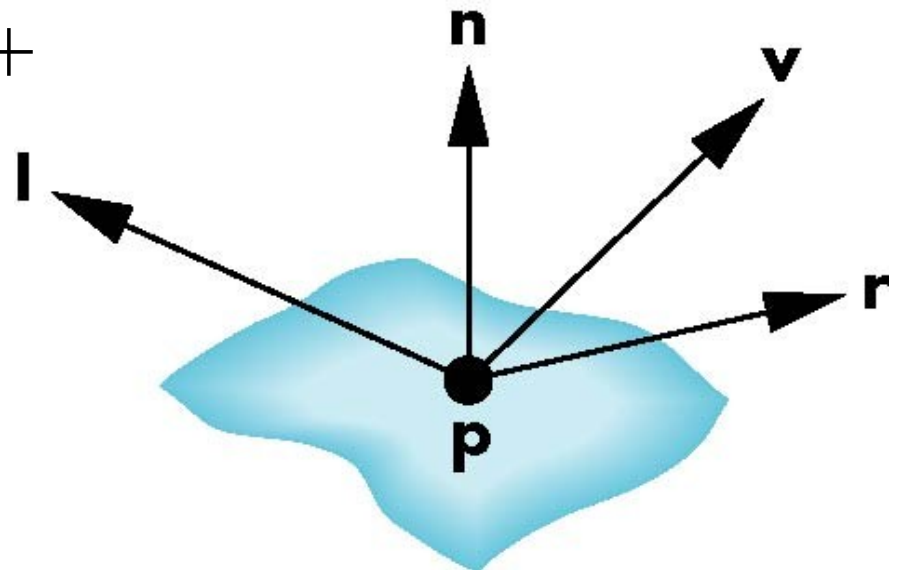


Phong Model

For each light source and each color component, the Phong model can be written (without the distance terms) as

$$I = k_d I_d \mathbf{l} \cdot \mathbf{n} + k_s I_s (\mathbf{v} \cdot \mathbf{r})^\alpha + k_a I_a$$

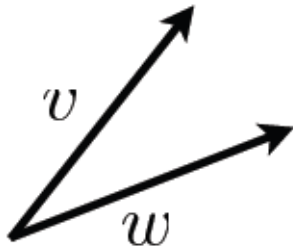
For each color component we add contributions from all sources



Overview

- Vector – Dot Product
- Other useful things with dot products

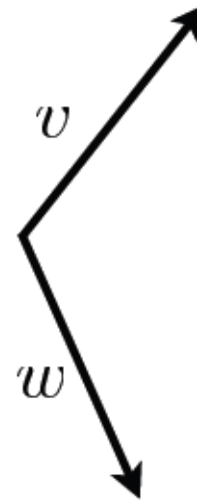
$$v \cdot w$$



$$v \cdot w > 0$$



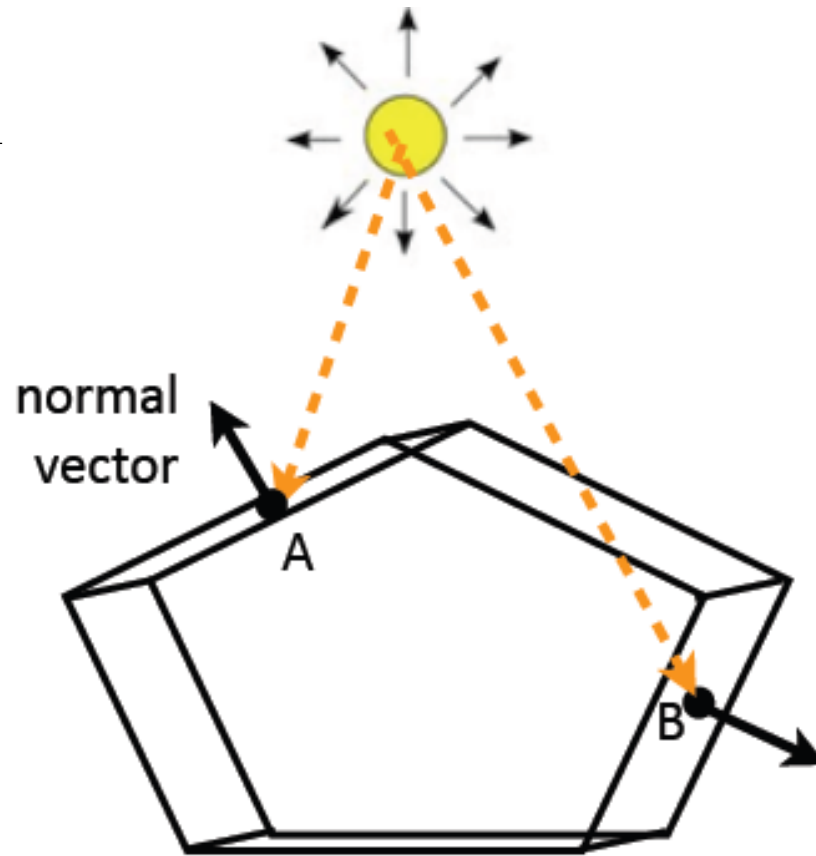
$$v \cdot w = 0$$



$$v \cdot w < 0$$

Overview

- Vector – Dot Product
- Determining “front” vs. “back” facing triangles
 - Back-face cul



Overview

- Vector – Dot Product
- This is important because if I want to know the angle between two vectors, you can trivially compute:

$$u \cdot v = \|u\| \|v\| \cos(\theta)$$

$$\cos(\theta) = \frac{u \cdot v}{\|u\| \|v\|}$$

$$\theta = \cos^{-1}\left(\frac{u \cdot v}{\|u\| \|v\|}\right)$$

- Note that if both u and v are unit vectors, then:

$$\theta = \cos^{-1}(u \cdot v)$$

- Note that θ will be in radians (that is, $0 \leq \theta \leq \pi$)

Overview

- Vector – Cross Product
- More formally: the cross product of $v = \begin{bmatrix} v_x \\ v_y \\ v_z \end{bmatrix}$ and $u = \begin{bmatrix} u_x \\ u_y \\ u_z \end{bmatrix}$ is written as:

$$v \times u = \begin{bmatrix} v_x \\ v_y \\ v_z \end{bmatrix} \times \begin{bmatrix} u_x \\ u_y \\ u_z \end{bmatrix}, \text{ and is defined}$$

as:

$$\begin{bmatrix} v_x \\ v_y \\ v_z \end{bmatrix} \times \begin{bmatrix} u_x \\ u_y \\ u_z \end{bmatrix} = \begin{bmatrix} v_y u_z - v_z u_y \\ v_z u_x - v_x u_z \\ v_x u_y - v_y u_x \end{bmatrix}$$

Overview - Computation: $A \times B = C$

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \quad [2 \times 2]$$

$$B = \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \end{bmatrix} \quad [2 \times 3]$$

$$C = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} & a_{11}b_{13} + a_{12}b_{23} \\ a_{21}b_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} & a_{21}b_{13} + a_{22}b_{23} \end{bmatrix} \quad [2 \times 3]$$

Overview - Computation: $A \times B = C$

$$A = \begin{bmatrix} 2 & 3 \\ 1 & 1 \\ 1 & 0 \end{bmatrix} \text{ and } B = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 0 & 2 \end{bmatrix}$$

$[3 \times 2]$ $[2 \times 3]$

A and B can be multiplied

$$C = \begin{bmatrix} 2*1+3*1=5 & 2*1+3*0=2 & 2*1+3*2=8 \\ 1*1+1*1=2 & 1*1+1*0=1 & 1*1+1*2=3 \\ 1*1+0*1=1 & 1*1+0*0=1 & 1*1+0*2=1 \end{bmatrix} = \begin{bmatrix} 5 & 2 & 8 \\ 2 & 1 & 3 \\ 1 & 1 & 1 \end{bmatrix}$$

$[3 \times 3]$

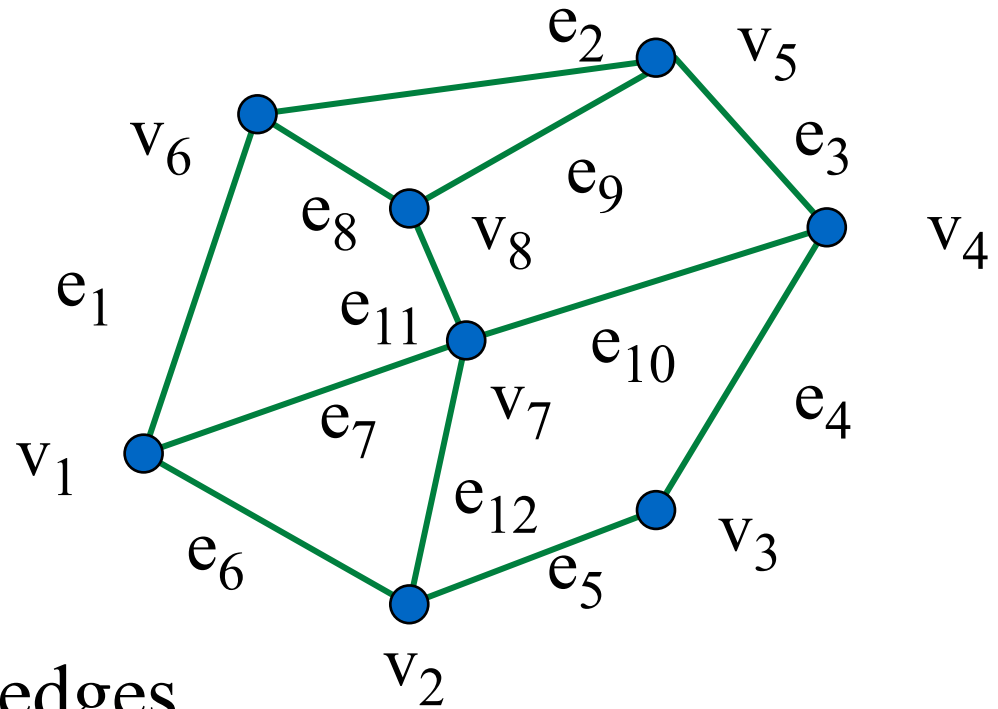
Overview (Concepts)

- dot product;
- cross product;
- matrix multiplication;
- triangle normal;
- homogeneous coordinates;
- orthogonal (parallel) projection; perspective projection;
- vector normalization; graphics pipeline;
- viewing frustum;
- composing transform;
- transform: rotate, scale, translation;

Models and Modeling

Object (consist of many Ps)

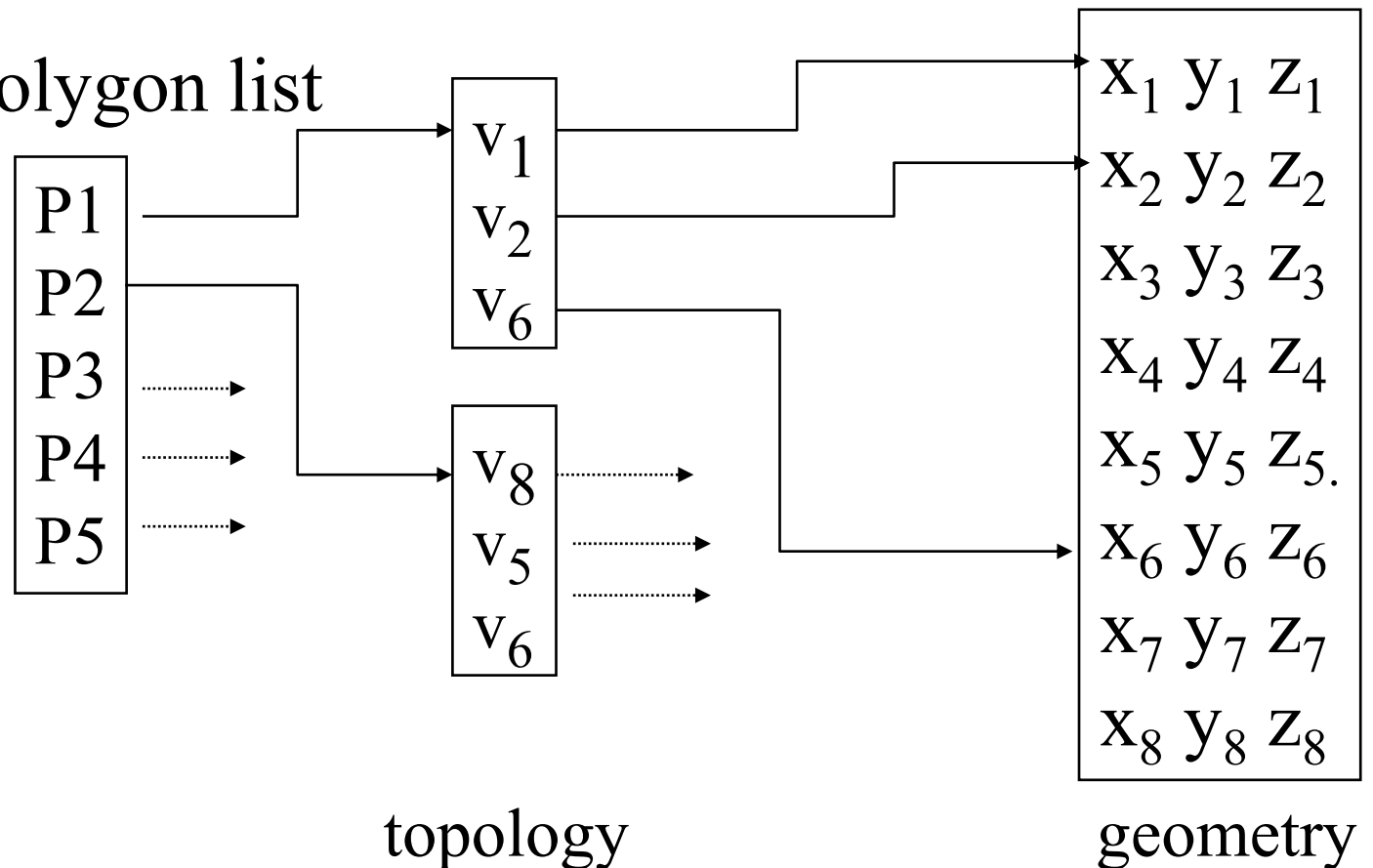
- Consider a mesh



- There are 8 nodes and 12 edges
 - 5 interior polygons
 - 6 interior (shared) edges
- Each vertex has a location $v_i = (x_i \ y_i \ z_i)$

Object

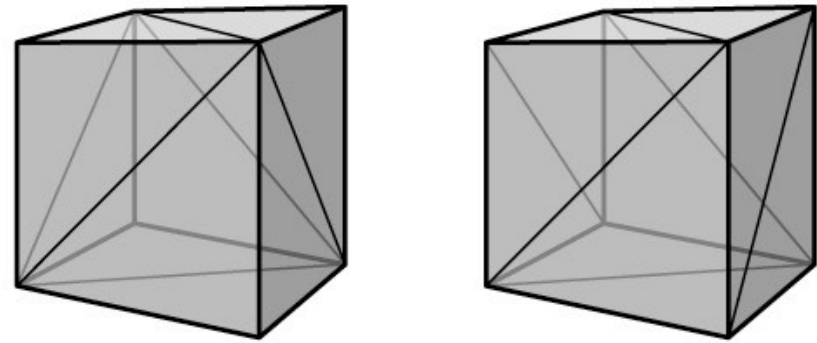
- Put the geometry in an array
- Use pointers from the vertices into this array
- Introduce a polygon list



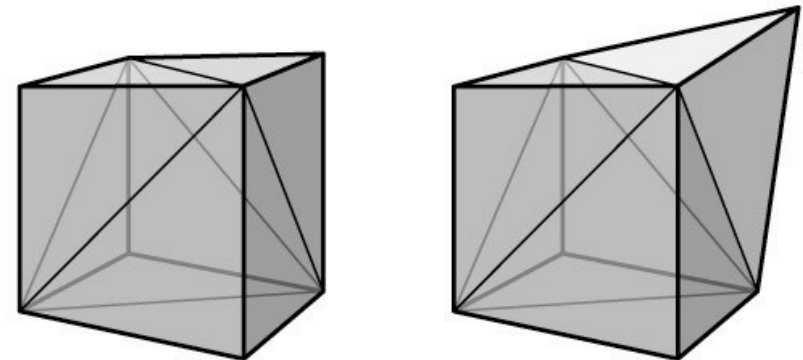
Object - Geometry vs Topology

- Generally it is a good idea to look for data structures that separate the geometry from the topology
 - Geometry: locations of the vertices
 - Topology: organization of the vertices and edges
 - Example: a polygon is an ordered list of vertices with an edge connecting successive pairs of vertices and the last to the first
 - Topology holds even if geometry changes

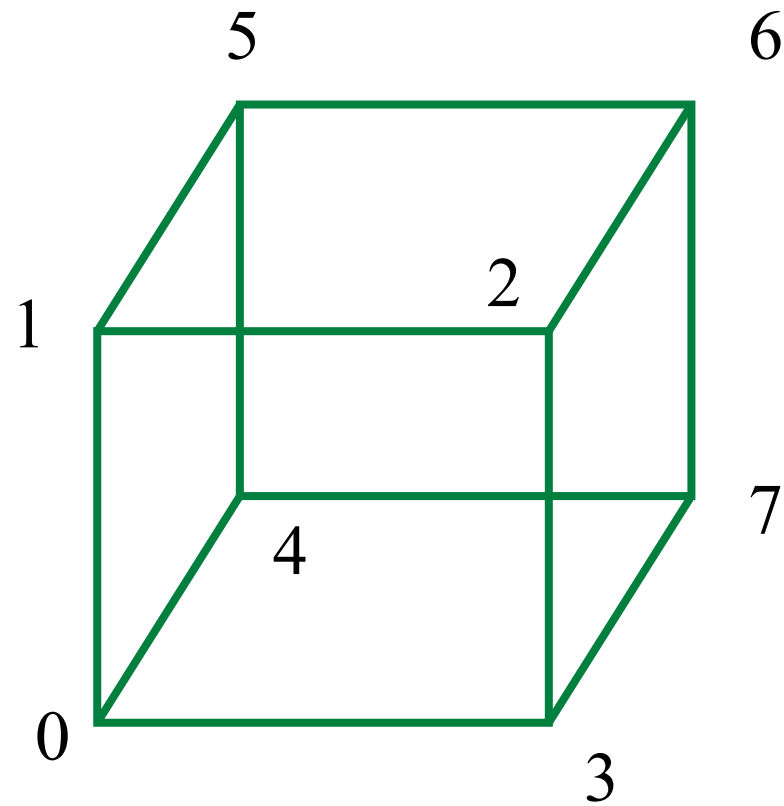
Same Geometry different topology



Same topology different geometry



Object - Cube



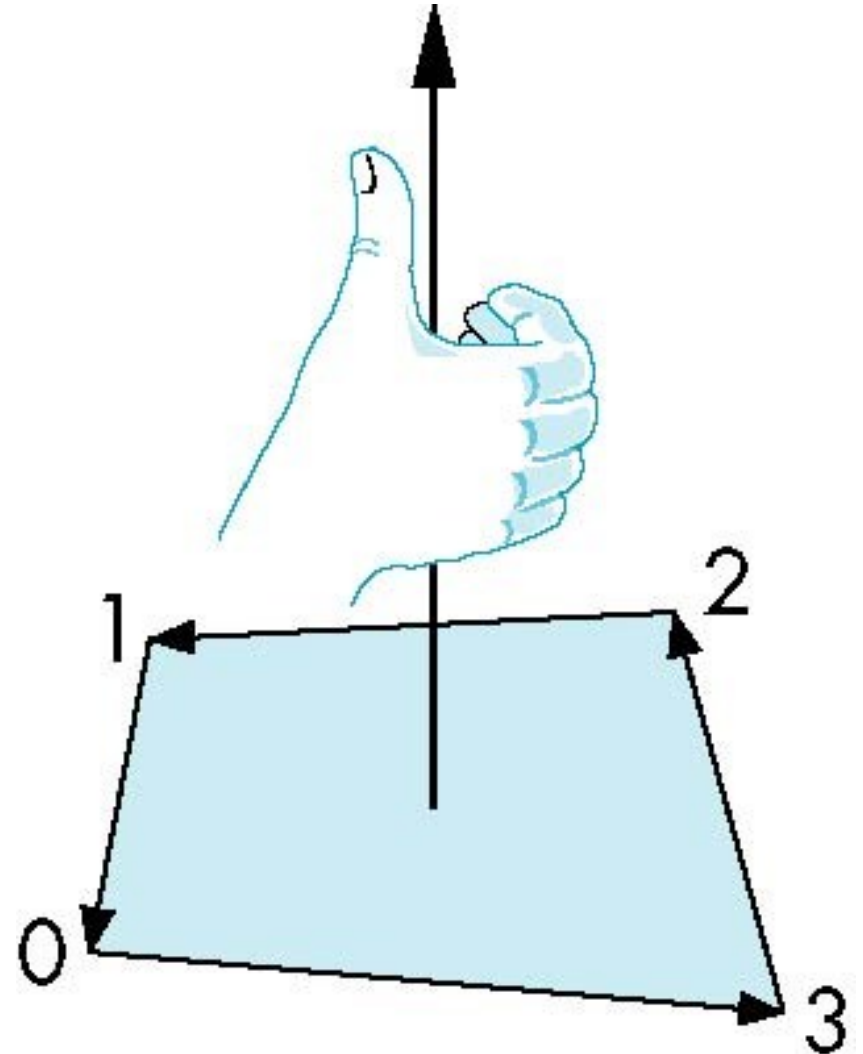
Object - Cube

Define global array for vertices

```
var vertices = [  
    vec3( -0.5, -0.5, 0.5 ),  
    vec3( -0.5, 0.5, 0.5 ),  
    vec3( 0.5, 0.5, 0.5 ),  
    vec3( 0.5, -0.5, 0.5 ),  
    vec3( -0.5, -0.5, -0.5 ),  
    vec3( -0.5, 0.5, -0.5 ),  
    vec3( 0.5, 0.5, -0.5 ),  
    vec3( 0.5, -0.5, -0.5 )  
];
```

Object - Cube

```
var indices = [  
  1,0,3,  
  3,2,1,  
  2,3,7,  
  7,6,2,  
  3,0,4,  
  4,7,3,  
  6,5,1,  
  1,2,6,  
  4,5,6,  
  6,7,4,  
  5,4,0,  
  0,1,5  
];
```



Transformations 3D – Model Matrix

Model * P

$$\text{Model} = \begin{bmatrix} 2 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$P = \begin{bmatrix} 4 & -4 \\ 6 & -6 \\ 2 & -2 \\ 1 & 1 \end{bmatrix}$$

Transformations 3D – Model Matrix

$$\text{Model} \cdot P = \begin{bmatrix} 2 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 2 & -2 \\ 2 & -2 \\ 2 & -2 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 4 & -4 \\ 6 & -6 \\ 2 & -2 \\ 1 & 1 \end{bmatrix}$$

More with Transformations

- Placing sub-objects in parent's coordinate system to construct hierarchical scene graph
 - Transforming primitives in their own coordinate systems
- View volume normalization
 - Mapping arbitrary view volume into canonical view volume along z-axis
- Parallel (orthographic, oblique) and perspective projections
- Perspective transformation (turn viewing pyramid into a cuboid to turn perspective projection into parallel projection) after perspective foreshortening

Composition of Transformations (2D)

30

- We now have a number of tools at our disposal; we can combine them!
- An object in a scene uses many transformations in sequence. How do we represent this in terms of functions?
- Transformation is a function; by associativity, we can compose functions:
 - $(f \circ g)(i)$
- This is the same as first applying g , then applying f :
 - $f(g(i))$
- Consider our functions f and g as matrices (M_1 and M_2) and our input as a vector v
- Our composition is equivalent to $M_1 M_2 v$

Composition of Transformations (2D)

31

- We can now form compositions of transformation matrices to form a more complex transformation
- For example, ***TRS*** $\mathbf{v}$, which scales a point, then rotates it, then translates it:
- Note that we apply the matrices in sequence right to left. We can use associativity to compose them first; it is often useful to be able to apply a single matrix if, for example, we want to use it to transform many points at once.
- **Important: order matters!** Matrix multiplication is **NOT commutative**.

- \[\begin{bmatrix} 1 & 0 & dx \\ 0 & 1 & dy \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \]

Transformations 3D – Model Matrix

32

Transformation	Matrix	Comments
Scaling	$\begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$	Looks just like the 2D version. We just added an s_z term.
Rotation	(see next slide)	In 2D, only one axis of rotation; now there are infinitely many! Must take all into account. See next slide...
Translation	$\begin{bmatrix} 1 & 0 & 0 & dx \\ 0 & 1 & 0 & dy \\ 0 & 0 & 1 & dz \\ 0 & 0 & 0 & 1 \end{bmatrix}$	Similar to the 2D version, just with one more entry dz , representing change in the z -direction.

Rotating axis by axis (1/2)

$R_{yz}(\psi)$: rotation about x axis by ψ	$R_{zx}(\theta)$: rotation about y axis by θ	$R_{xy}(\phi)$: rotation about z axis by ϕ
$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \psi & -\sin \psi & 0 \\ 0 & \sin \psi & \cos \psi & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$	$\begin{bmatrix} \cos \phi & -\sin \phi & 0 & 0 \\ \sin \phi & \cos \phi & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

Inverses

Transformation	Inverse Matrix
Scaling	$\begin{bmatrix} 1/s_x & 0 & 0 & 0 \\ 0 & 1/s_y & 0 & 0 \\ 0 & 0 & 1/s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$
Rotation	$\begin{matrix} R_{yz}^{-1}(\psi) & R_{zx}^{-1}(\theta) & R_{xy}^{-1}(\phi) \end{matrix}$ $\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \psi & \sin \psi & 0 \\ 0 & -\sin \psi & \cos \psi & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} \cos \theta & 0 & -\sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ \sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \begin{bmatrix} \cos \phi & \sin \phi & 0 & 0 \\ -\sin \phi & \cos \phi & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$
Translation	$\begin{bmatrix} 1 & 0 & 0 & -dx \\ 0 & 1 & 0 & -dy \\ 0 & 0 & 1 & -dz \\ 0 & 0 & 0 & 1 \end{bmatrix}$

Example in 3D!

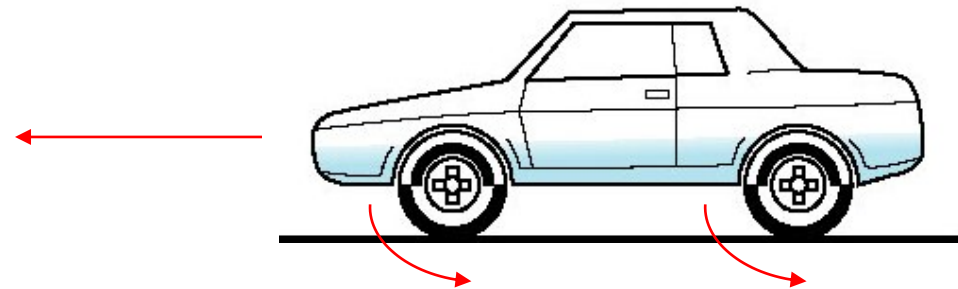
35

- Let's take some 3D object, say a cube centered at (2,2,2)
- Rotate in object's space by 30° around x axis, 60° around y , and 90° around z
- Scale in object space by 1 in the x , 2 in the y , 3 in the z
- Translate by (2,2,4) in world space
- Transformation sequence: $TT_0^{-1}S_{xy}R_{xy}R_{zx}R_{yz}T_o$, where T_o translates to origin:

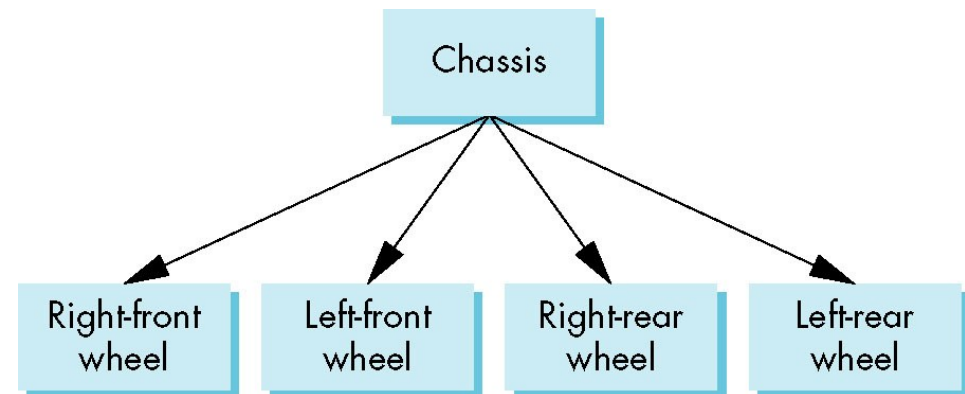
$$\begin{array}{ccccccc}
 \begin{bmatrix} 1 & 0 & 0 & 2 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 4 \\ 0 & 0 & 0 & 1 \end{bmatrix} & \begin{bmatrix} 1 & 0 & 0 & 2 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 1 \end{bmatrix} & \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} & \begin{bmatrix} \cos 90 & \sin 90 & 0 & 0 \\ -\sin 90 & \cos 90 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} & \begin{bmatrix} \cos 60 & 0 & -\sin 60 & 0 \\ 0 & 1 & 0 & 0 \\ \sin 60 & 0 & \cos 60 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} & \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos 30 & \sin 30 & 0 \\ 0 & -\sin 30 & \cos 30 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} & \begin{bmatrix} 1 & 0 & 0 & -2 \\ 0 & 1 & 0 & -2 \\ 0 & 0 & 1 & -2 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
 \mathbf{T} & \mathbf{T}_0^{-1} & \mathbf{S}_{xy} & \mathbf{R}_{xy} & \mathbf{R}_{zx} & \mathbf{R}_{yz} & \mathbf{T}_0
 \end{array}$$

Relationships in Car Model

- Symbol-instance table does not show relationships between parts of model
- Consider model of car
 - Chassis + 4 identical wheels
 - Two symbols/parts/subobjects
- Rate of forward motion determined by rotational speed of wheels

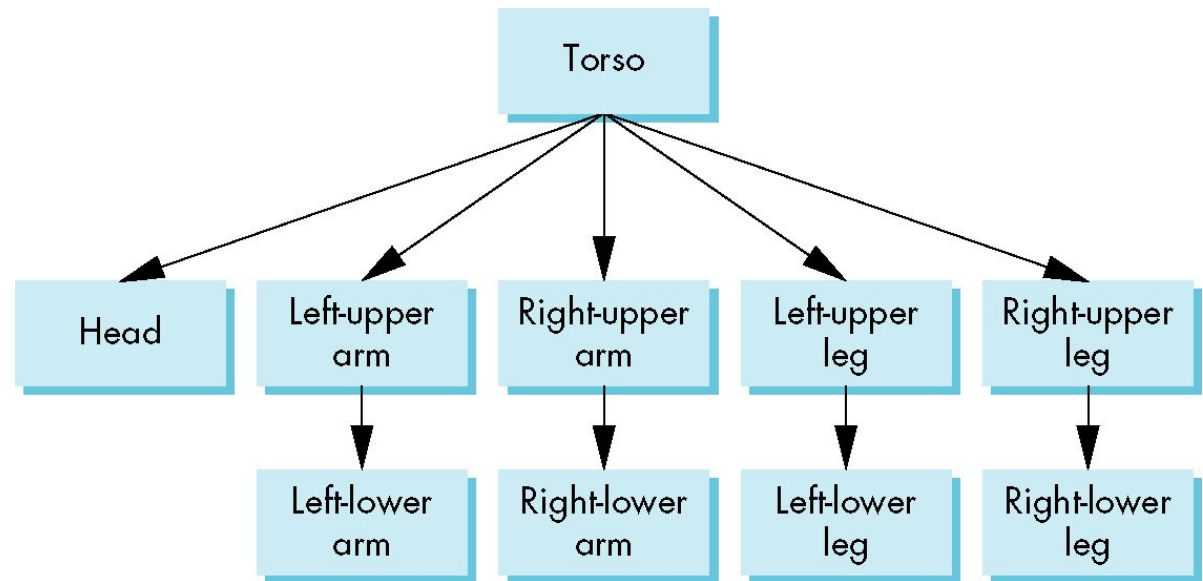
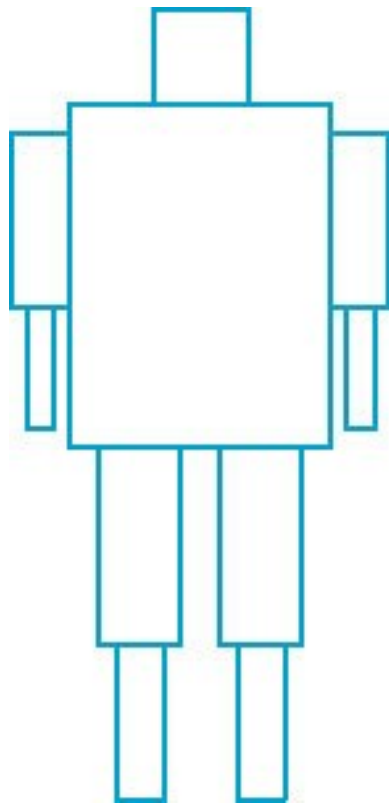


```
car (speed)
{
    chassis ()
    wheel(right_front);
    wheel(left_front);
    wheel(right_rear);
    wheel(left_rear);
}
```

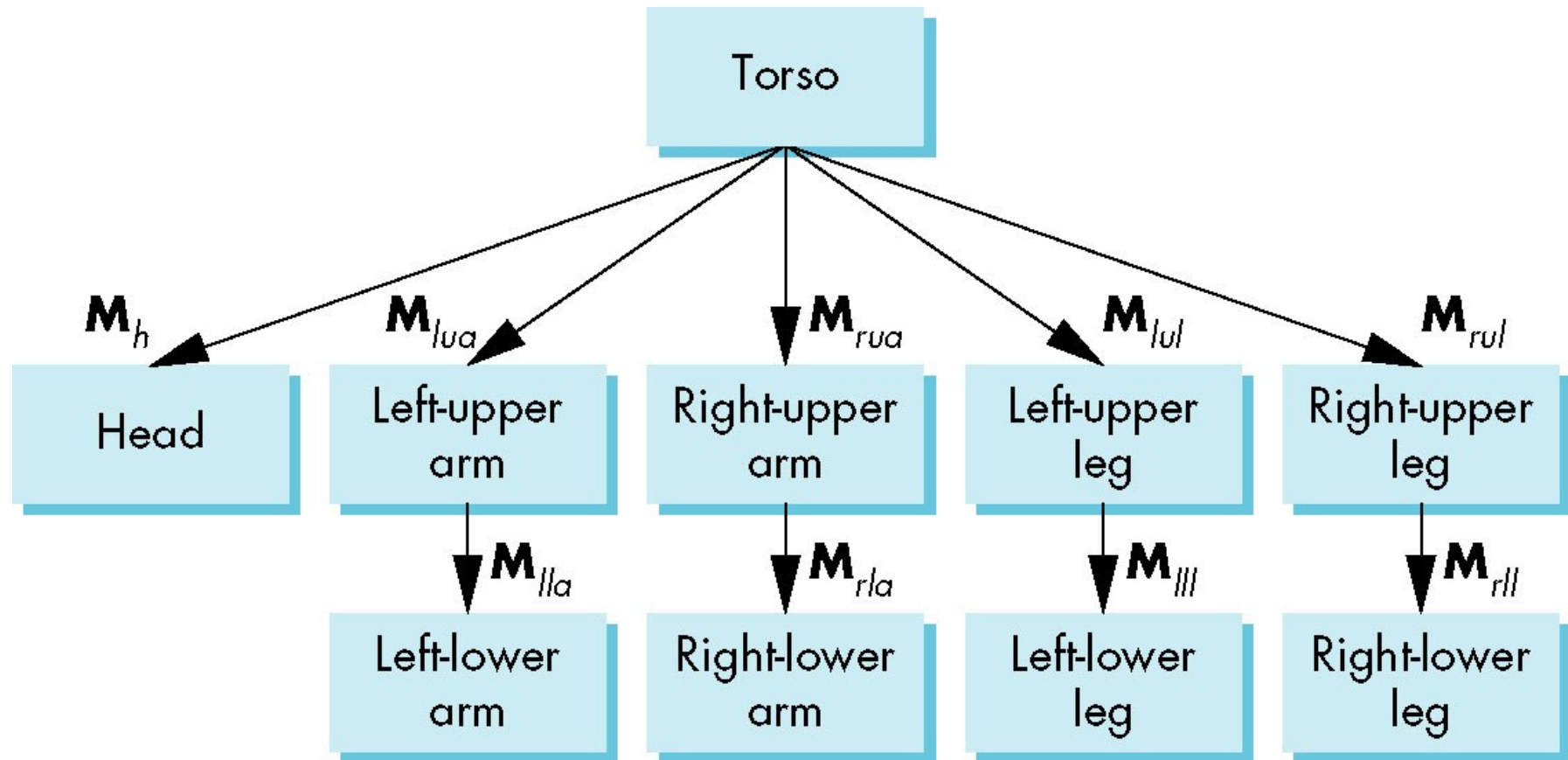


- Fails to show relationships well
- Look at problem using a graph

Humanoid Figure



Tree with Matrices



Display and Traversal

- The position of the figure is determined by 11 joint angles (two for the head and one for each other part)
- Display of the tree requires a *graph traversal*
 - Visit each node once
 - Display function at each node that describes the part associated with the node, applying the correct transformation matrix for position and orientation

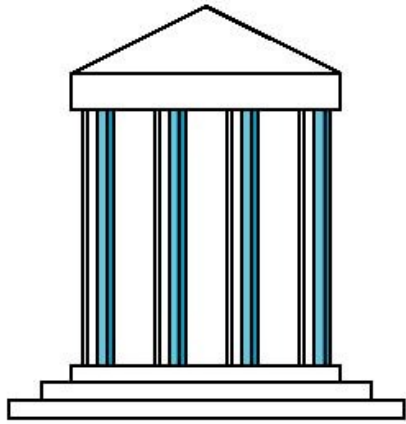
Traversal Code

```
figure() {  
    PushMatrix()      ← save present model-view matrix  
    torso();  
    Rotate(...)      ← update model-view matrix for head  
    head();  
    PopMatrix();      ← recover original model-view matrix  
    PushMatrix();      ← save it again  
    Translate(...);  
    Rotate(...)      ← update model-view matrix  
                        for left upper arm  
    left_upper_arm();  
    PopMatrix();      ← recover and save original  
                        model-view matrix again  
    PushMatrix();  
    ← rest of code
```

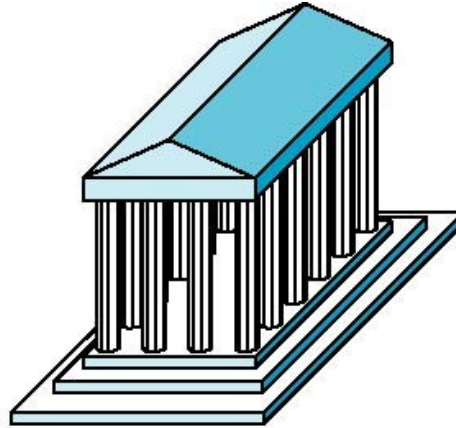
Viewing

Projection * View * Model * P

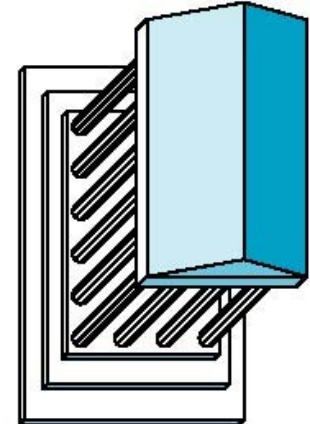
Classical Projections



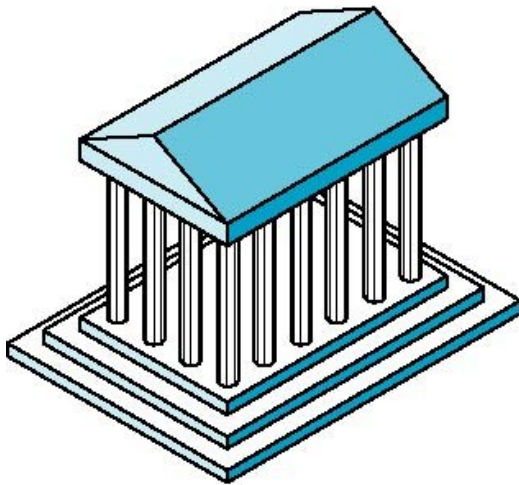
Front elevation



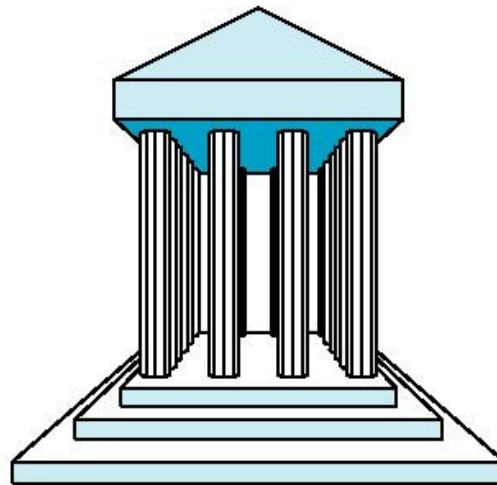
Elevation oblique



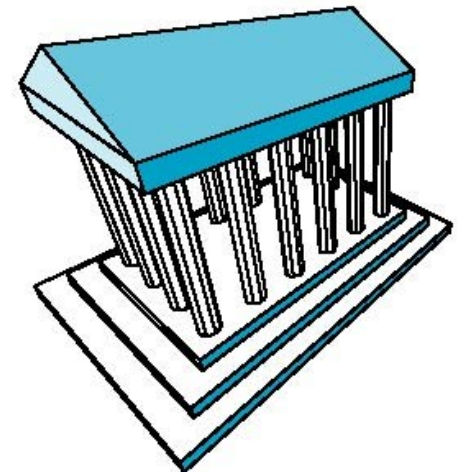
Plan oblique



Isometric



One-point perspective



Three-point perspective

Planar Geometric Projections

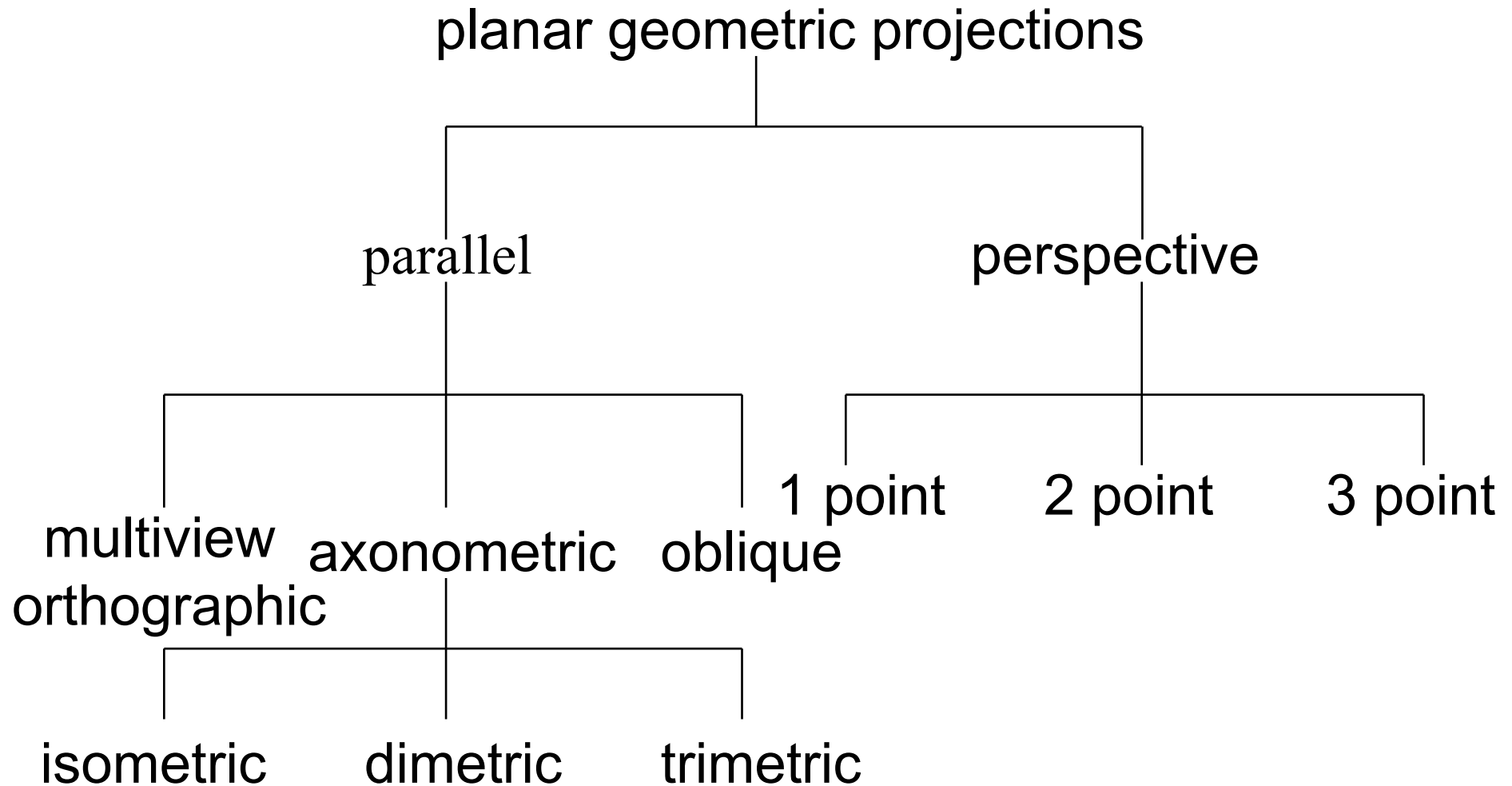
- Standard projections project onto a plane
- Projectors are lines that either
 - converge at a center of projection
 - are parallel
- Such projections preserve lines
 - but not necessarily angles
- Nonplanar projections are needed for applications such as map construction

Perspective vs Parallel

- Computer graphics treats all projections the same and implements them with a single pipeline
- Classical viewing developed different techniques for drawing each type of projection
- Fundamental distinction is between parallel and perspective viewing even though mathematically parallel viewing is the limit of perspective viewing

Taxonomy of Planar Geometric Projections

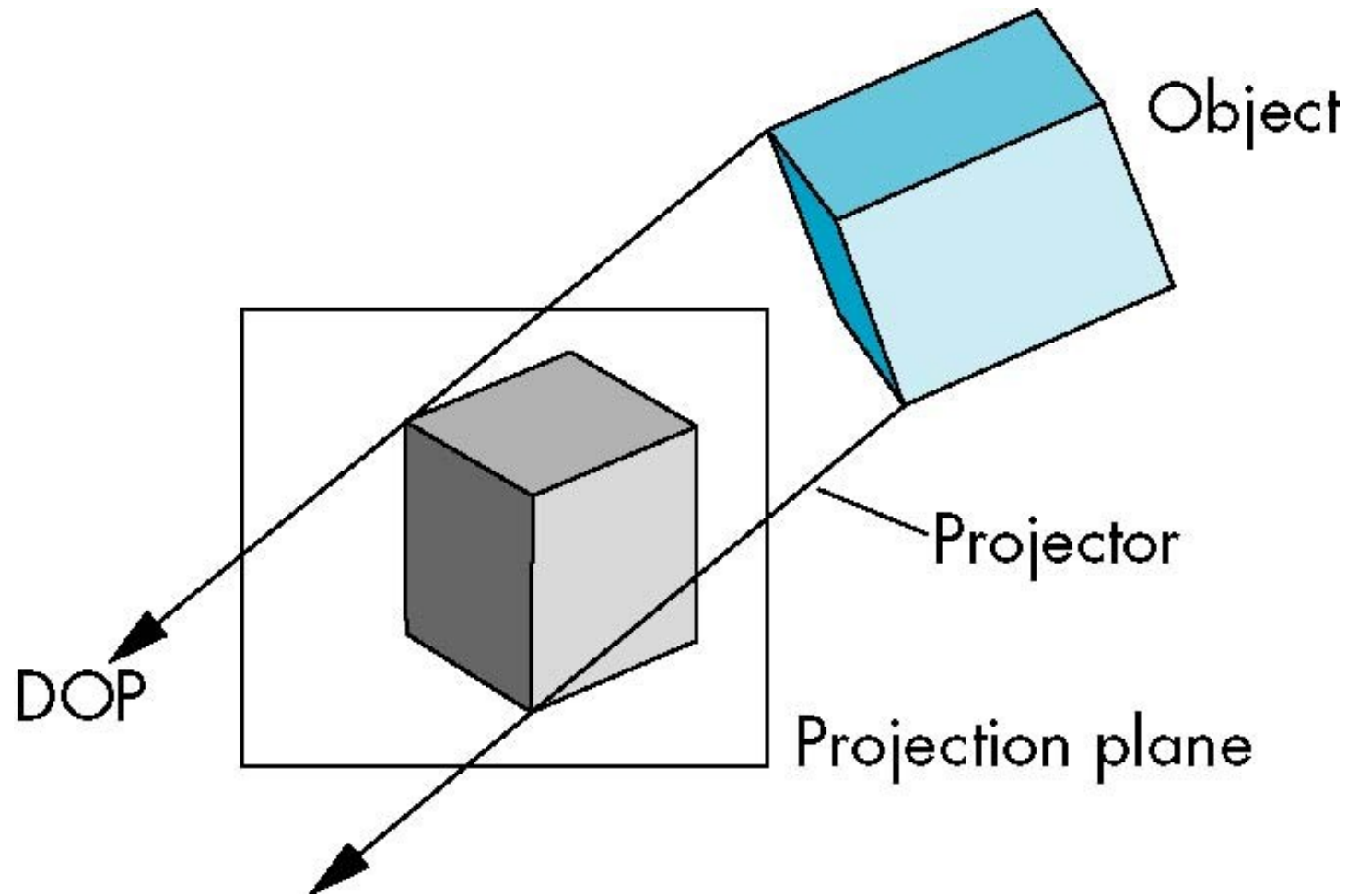
4
5



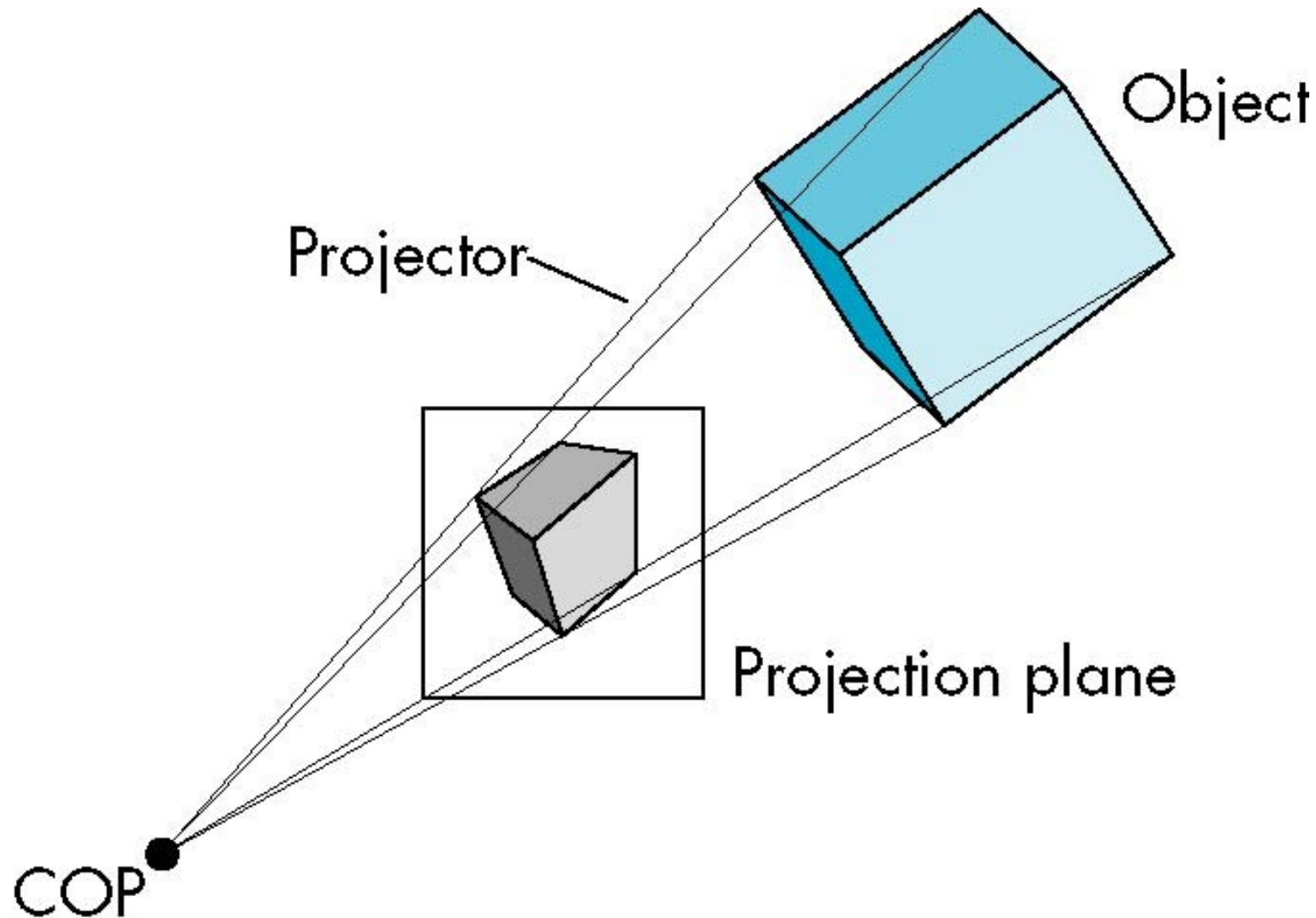
Taxonomy of Planar Geometric Projections

- Projectors are lines
- Projectors are parallel and perpendicular to the image plan one principal face (Multiview orthographic)
- Projectors are parallel and perpendicular to the image plan (axonometric)
- Projectors are parallel but not perpendicular to image plan (oblique)
- Projectors are not parallel and converge at COP (perspective)

Parallel Projection

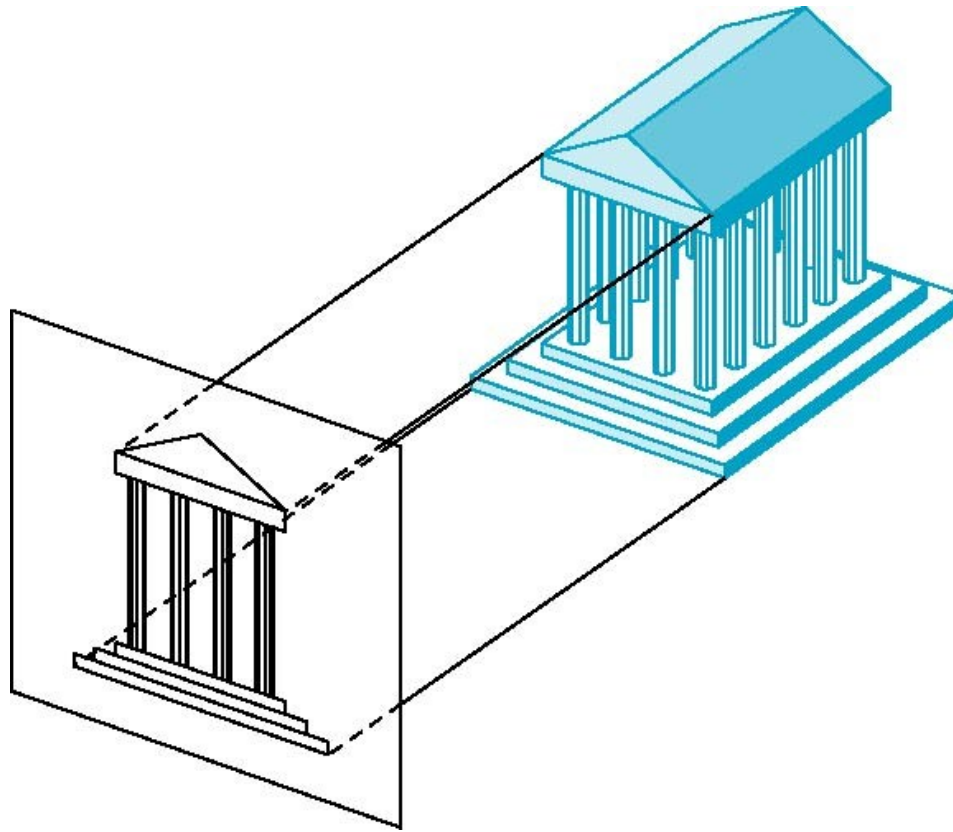


Perspective Projection



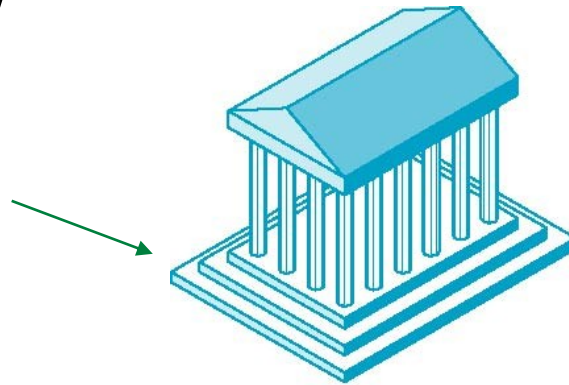
Orthographic Projection

Projectors are orthogonal to projection surface

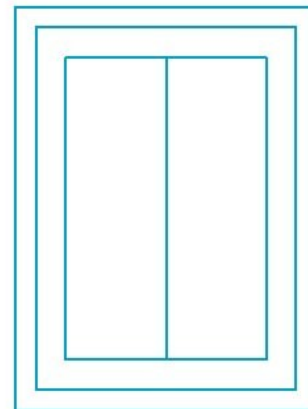


Multiview Orthographic Projection

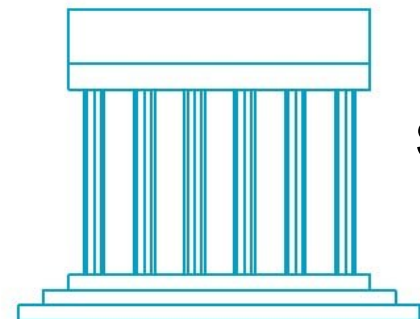
- Projection plane parallel to principal face
- Usually form front, top, side views isometric (not multiview orthographic view)



front



top



side

in CAD and architecture,
we often display three
multiviews plus isometric

Advantages and Disadvantages

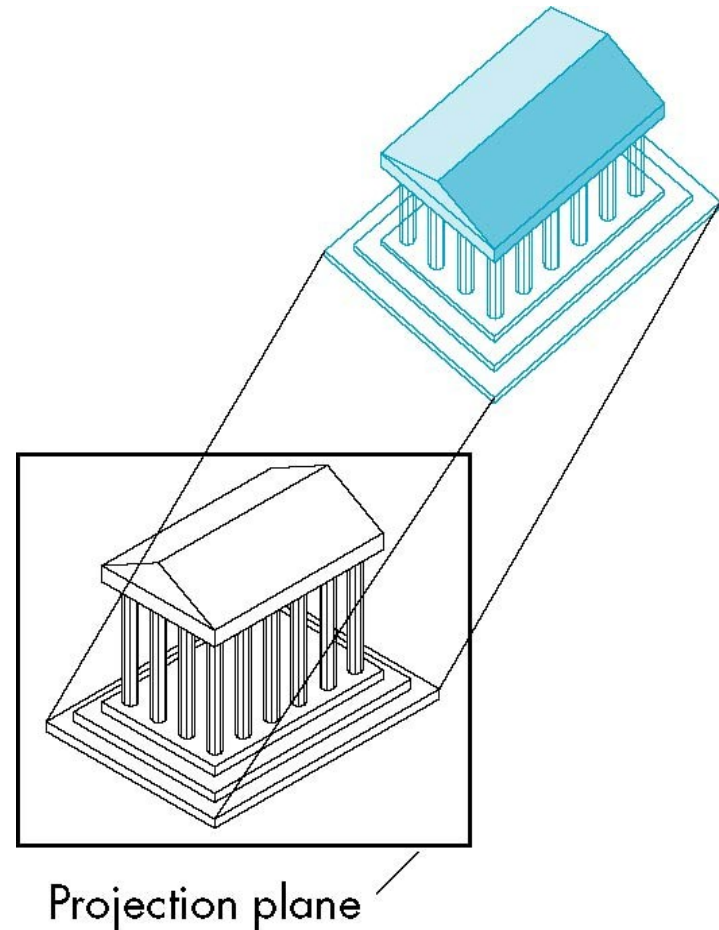
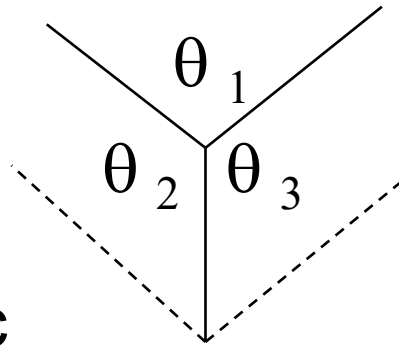
- Preserves both distances and angles
 - Shapes preserved
 - Can be used for measurements
 - Building plans
 - Manuals
- Cannot see what object really looks like because many surfaces hidden from view
 - Often we add the isometric

Axonometric Projections

Allow projection plane to move
relative to object

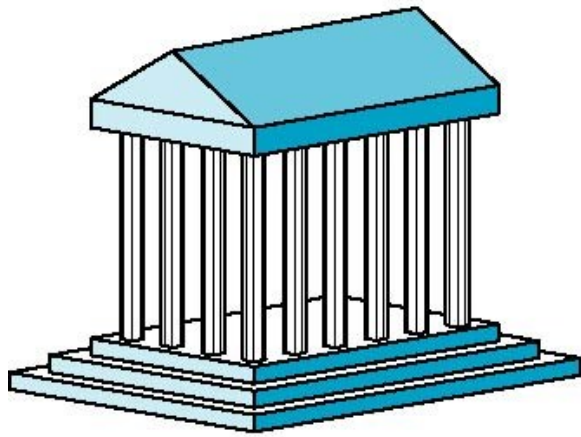
classify by how many angles of
a corner of a projected cube are
the same

none: trimetric
two: dimetric
three: isometric

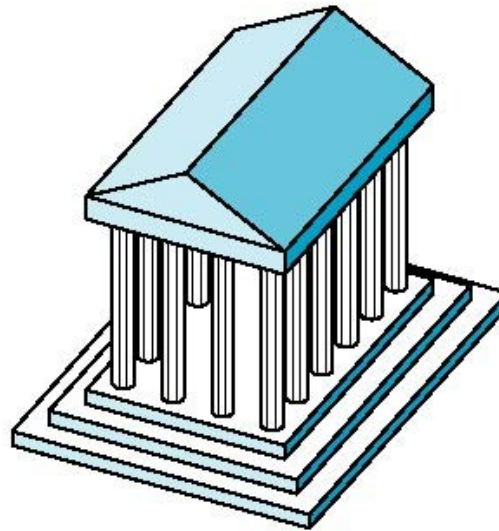


Types of Axonometric Projections

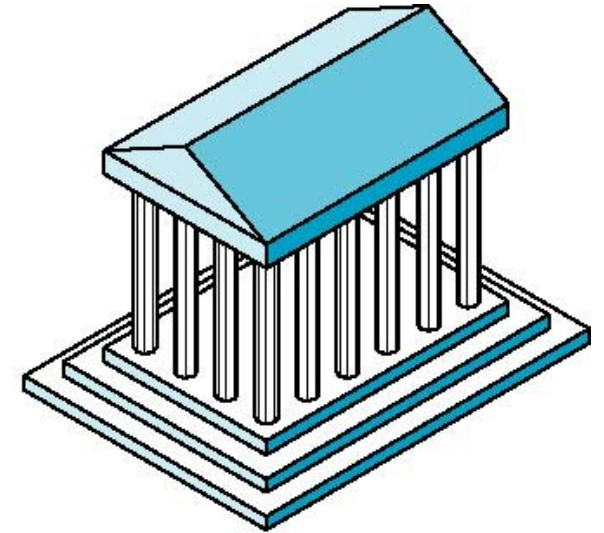
5
3



Dimetric



Trimetric



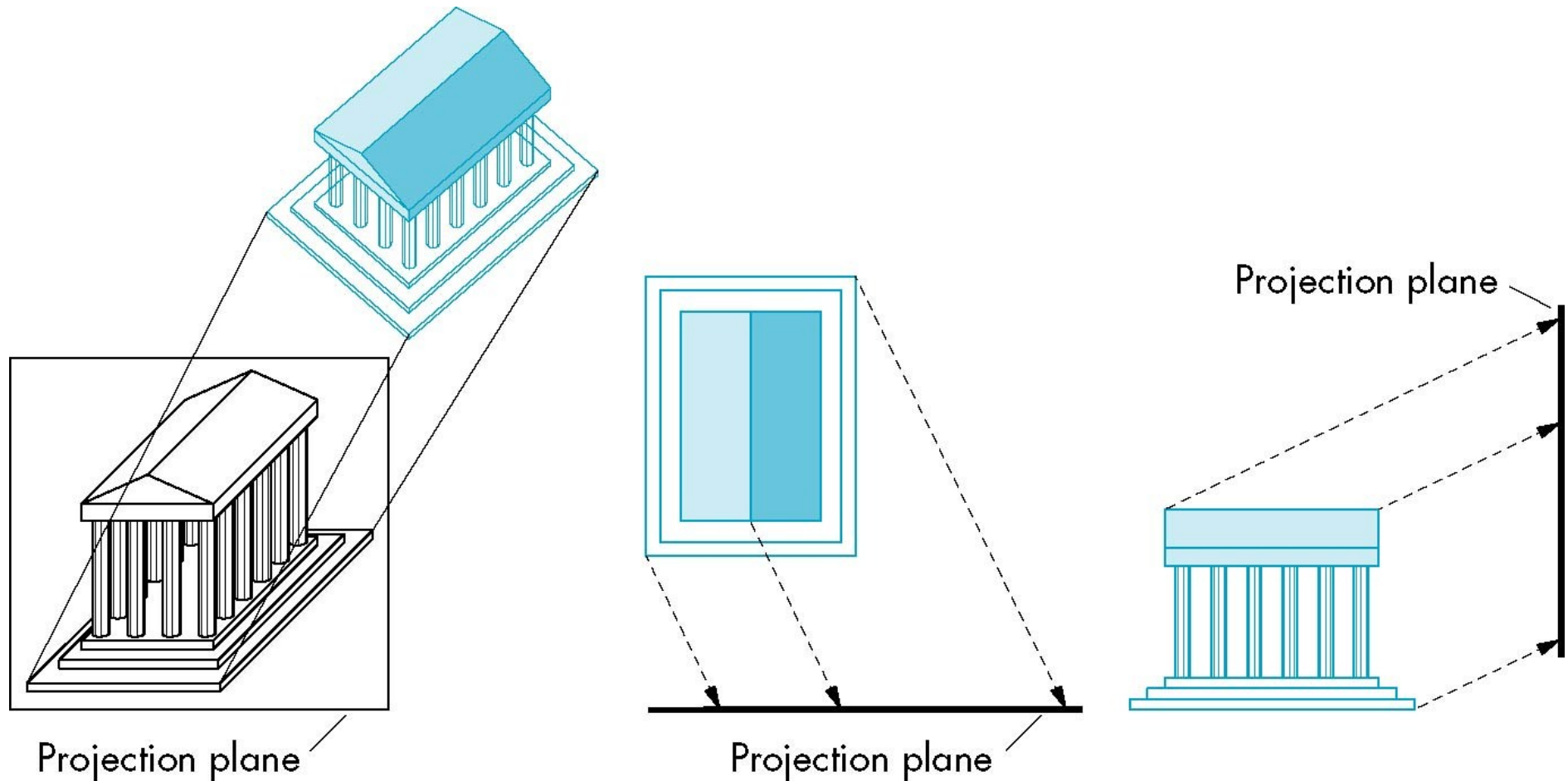
Isometric

Advantages and Disadvantages

- Lines are scaled (*foreshortened*) but can find scaling factors
- Lines preserved but angles are not
 - Projection of a circle in a plane not parallel to the projection plane is an ellipse
- Can see three principal faces of a box-like object
- Some optical illusions possible
 - Parallel lines appear to diverge
- Does not look real because far objects are scaled the same as near objects
- Used in CAD applications

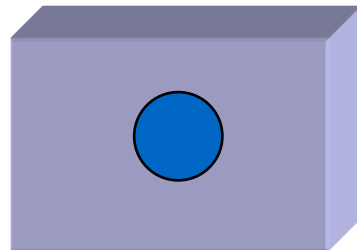
Oblique Projection

Arbitrary relationship between projectors and projection plane



Advantages and Disadvantages

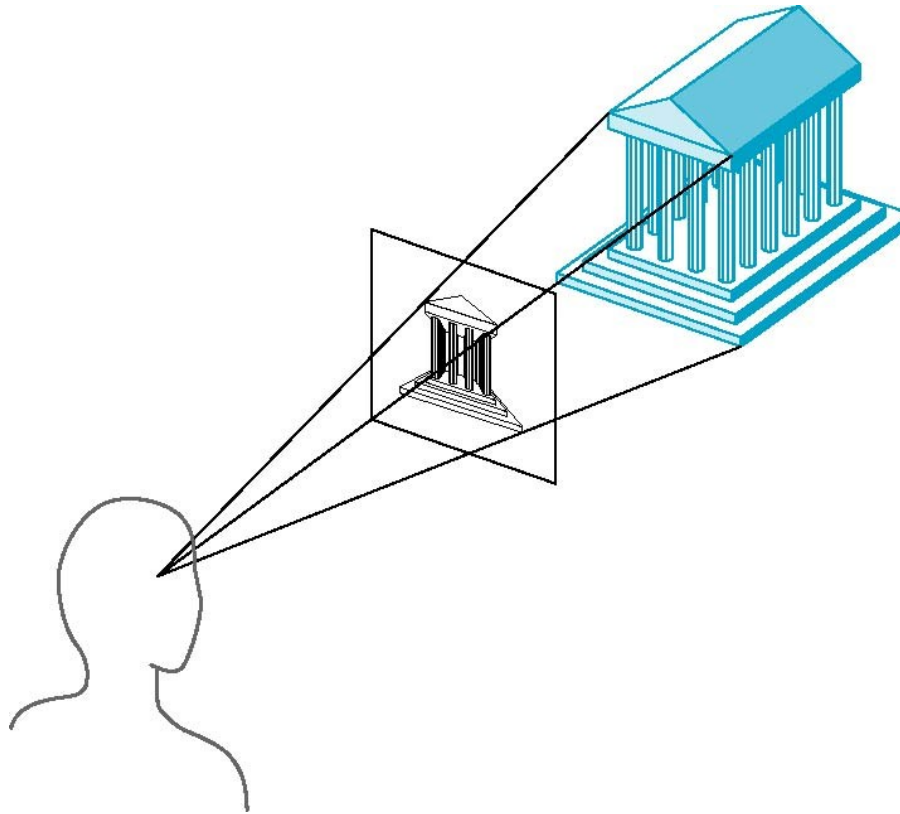
- Can pick the angles to emphasize a particular face
- Angles in faces parallel to projection plane are preserved while we can still see “around” side



- In physical world, cannot create with simple camera; possible with bellows camera or special lens (architectural)

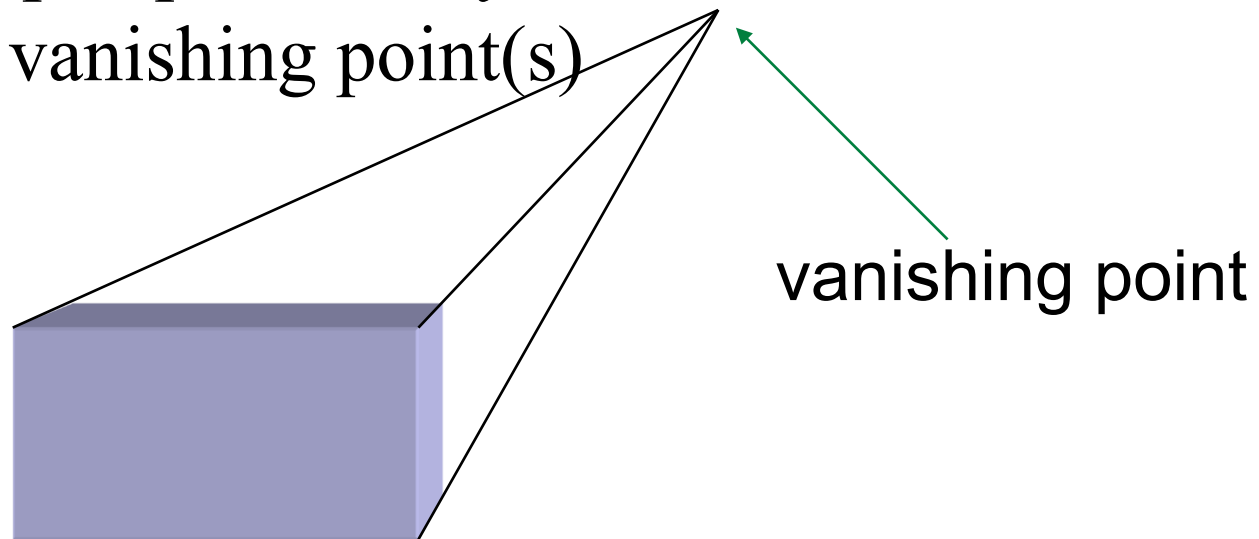
Perspective Projection

Projectors converge at center of projection



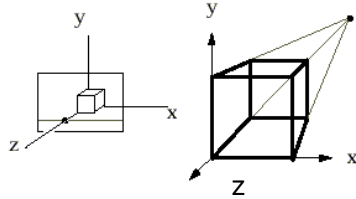
Vanishing Points

- Parallel lines (not parallel to the projection plan) on the object converge at a single point in the projection (the *vanishing point*)
- Drawing simple perspectives by hand uses these vanishing point(s)

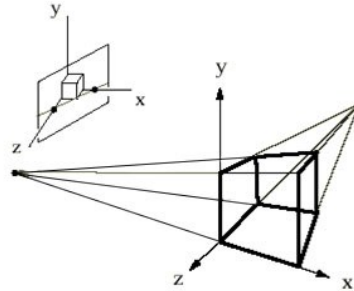


Vanishing Points

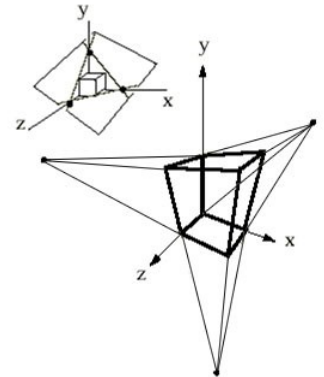
One Point Perspective
(z-axis vanishing point)



Two Point Perspective
(z and x-axis vanishing points)

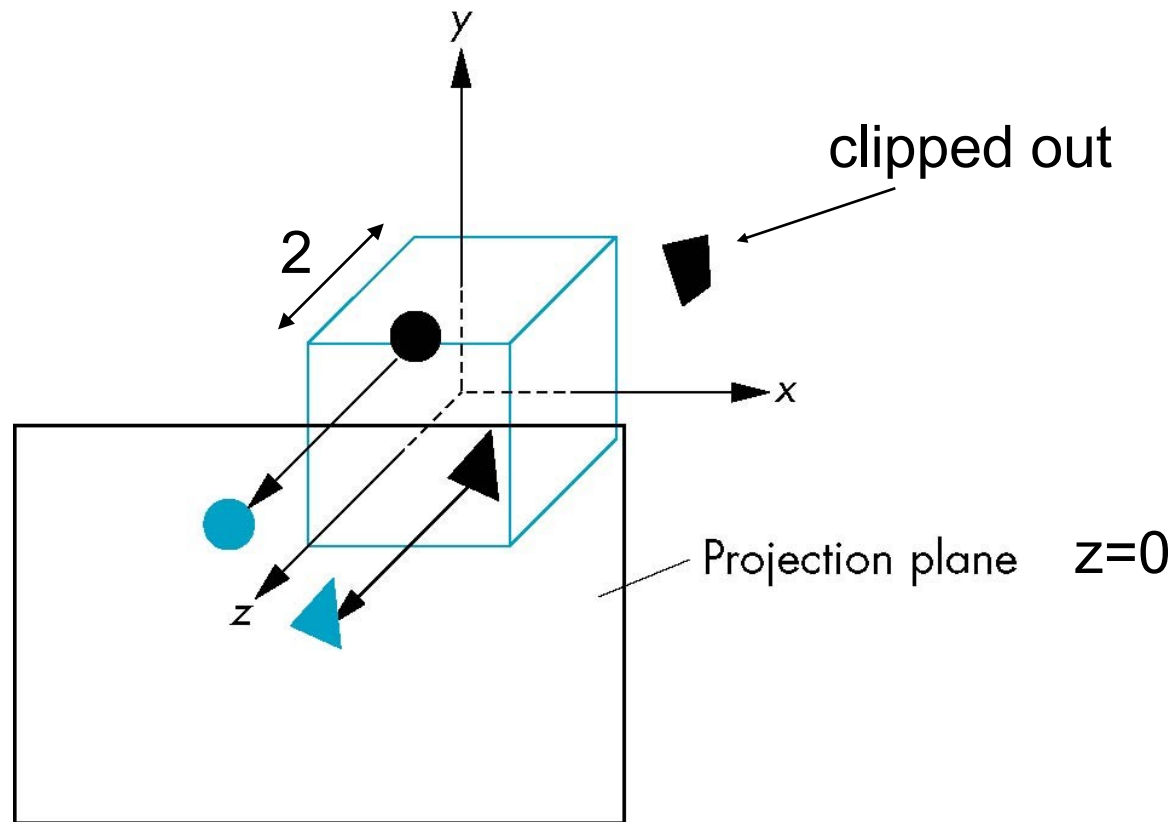


Three Point Perspective
(z, x, and y-axis vanishing points)



Default Projection

Default projection is orthogonal



View/Camera Placement

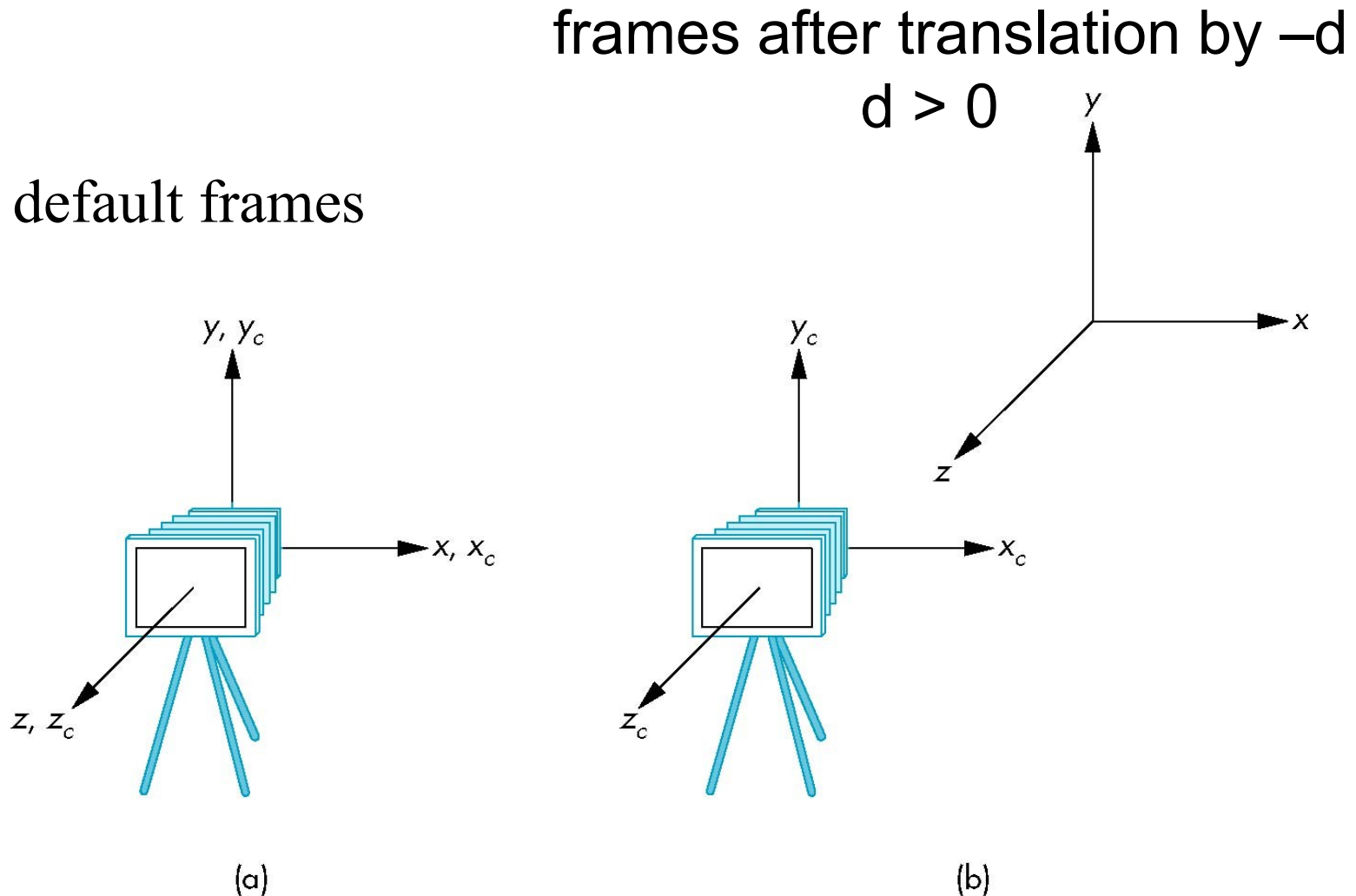
- Still 3D scene

Projection * View * Model * P

Moving the Camera Frame

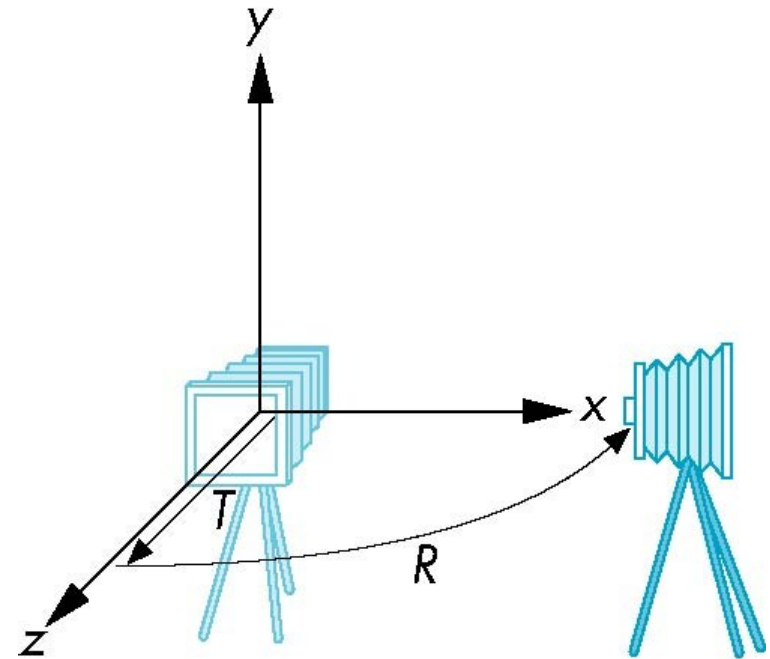
- If we want to visualize objects with both positive and negative z values we can either
 - Move the camera in the positive z direction
 - Translate the camera frame
 - Move the objects in the negative z direction
 - Translate the world frame
- Both of these views are equivalent and are determined by the model-view matrix
 - Want a translation (**`translate(0.0, 0.0, -d)`** ;)
 - **`d > 0`**

Moving Camera back from Origin



Moving the Camera

- We can move the camera to any desired position by a sequence of rotations and translations
- Example: side view
 - Rotate the camera
 - Move it away from origin
 - View matrix $\mathbf{C} = \mathbf{TR}$



WebGL code

- Remember that last transformation specified is first to be applied

```
// Using MV.js
```

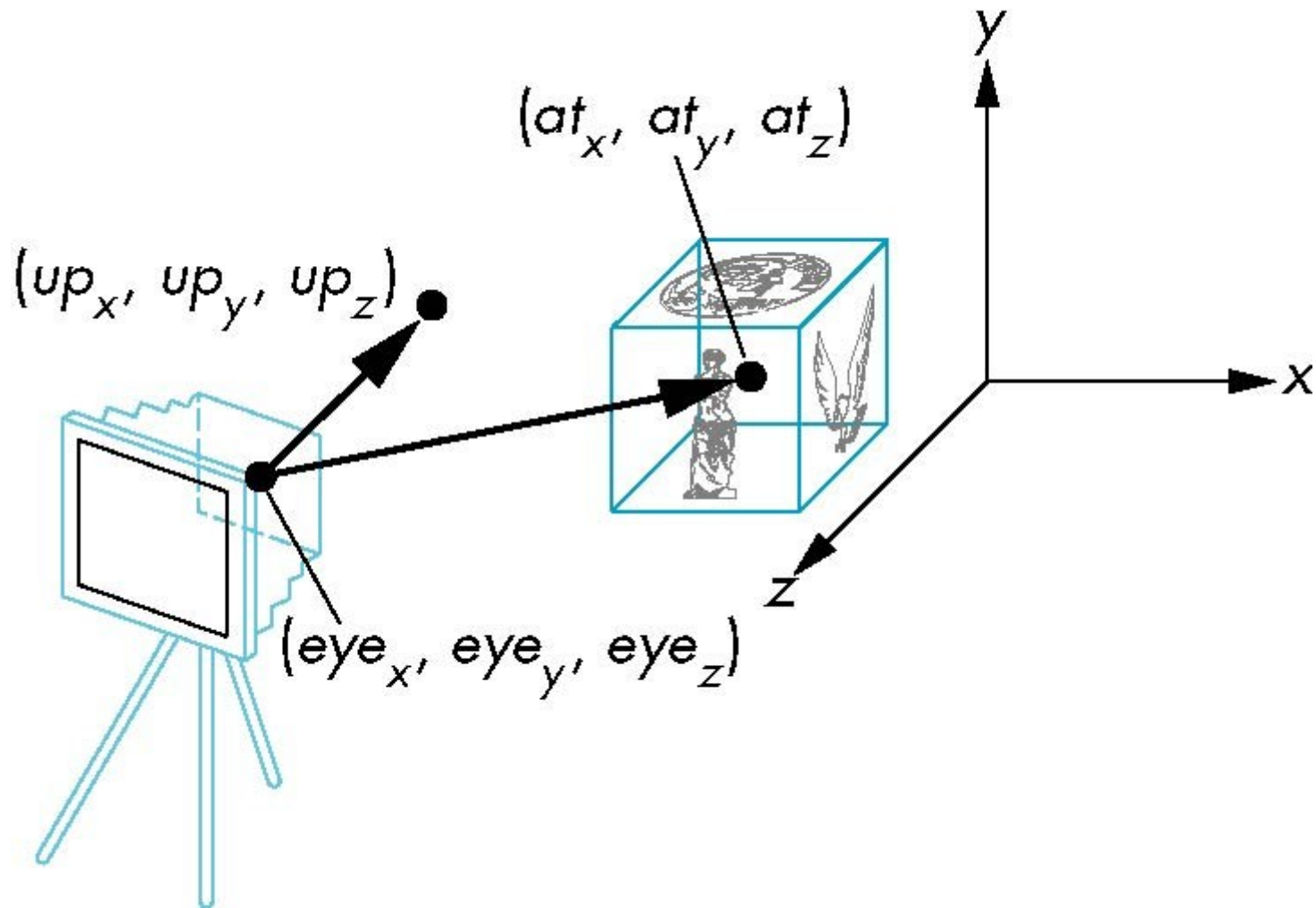
```
var t = translate (0.0, 0.0, -d) ;  
var ry = rotateY(90.0) ;  
var m = mult(t, ry) ;
```

or

```
var V = mult(translate (0.0, 0.0, -d) ,  
             rotateY(90.0) ;) ;
```

lookAt

`LookAt(eye, at, up)`



The lookAt Function

- The GLU library contained the function `gluLookAt` to form the required modelview matrix through a simple interface
- Note the need for setting an up direction
- Replaced by `lookAt()` in `MV.js`
 - Can concatenate with modeling transformations
- Example: isometric view of cube aligned with axes

```
var eye = vec3(1.0, 1.0, 1.0);  
var at = vec3(0.0, 0.0, 0.0);  
var up = vec3(0.0, 1.0, 0.0);  
  
var view = LookAt(eye, at, up);
```

Projection

- Going to 2D/image but preserve depth information Z for hidden surface removing
- Extract the x_p , y_p from x , y , z that will be displayed on the screen
- Map the world to native WebGL viewing volume

Projection * View * Model * P

Projections and Normalization

- The default projection in the eye (camera) frame is orthogonal
- For points within the default view volume

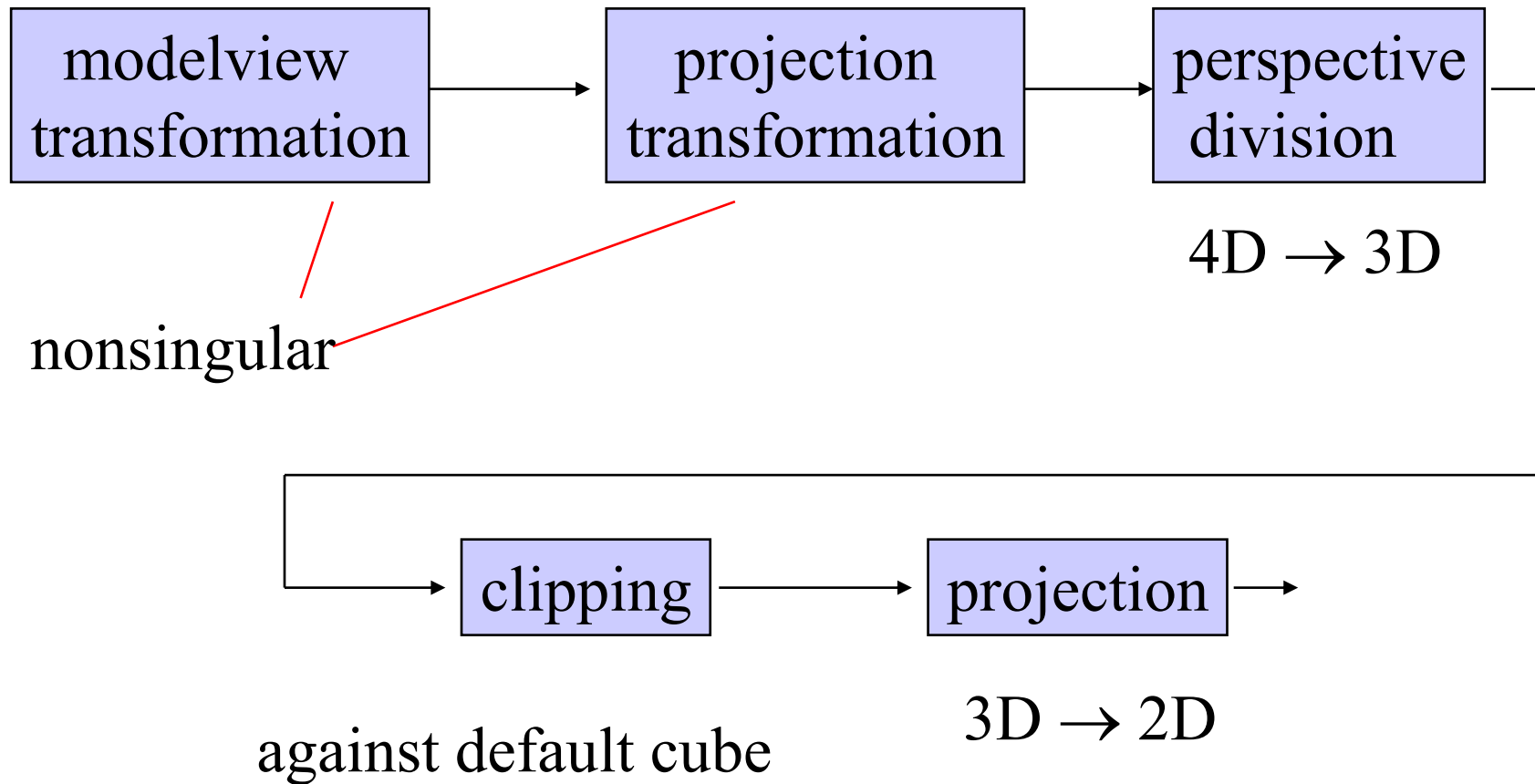
$$x_p = x$$

$$y_p = y$$

$$z_p = 0$$

- Most graphics systems use *view normalization*
 - All other views are converted to the default view by transformations that determine the projection matrix
 - Allows use of the same pipeline for all views

Pipeline View



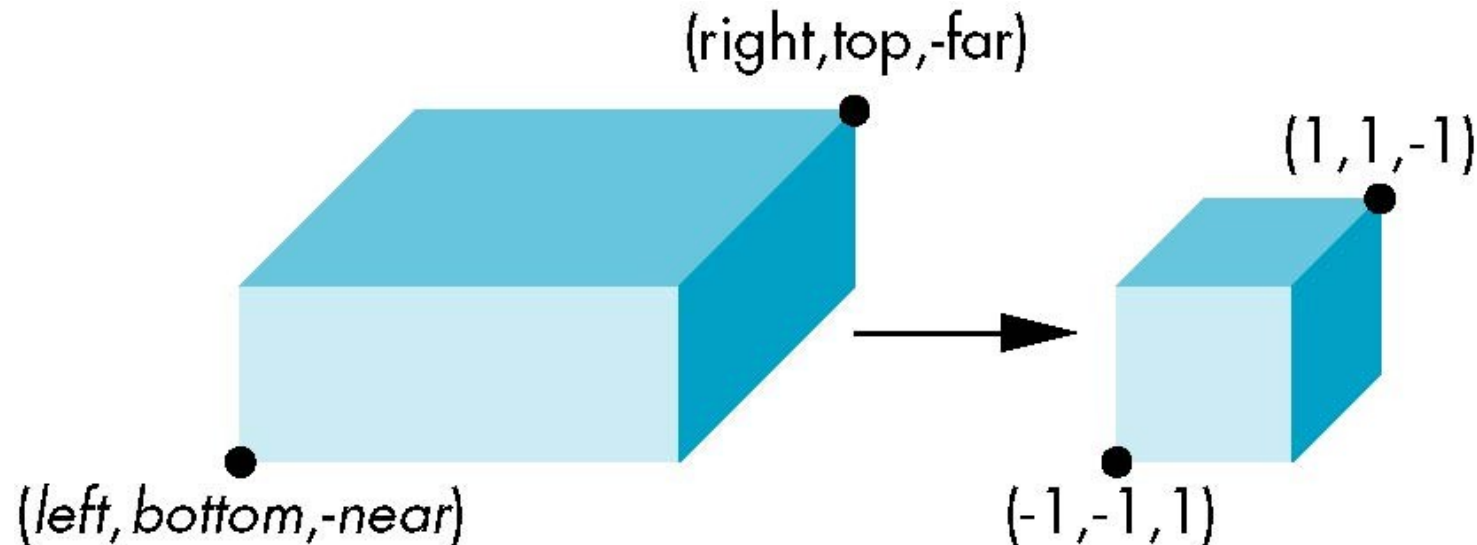
Notes

- We stay in four-dimensional homogeneous coordinates through both the modelview and projection transformations
 - Both these transformations are nonsingular
 - Default to identity matrices (orthogonal view)
- Normalization lets us clip against simple cube regardless of type of projection
- Delay final projection until end
 - Important for hidden-surface removal to retain depth information as long as possible

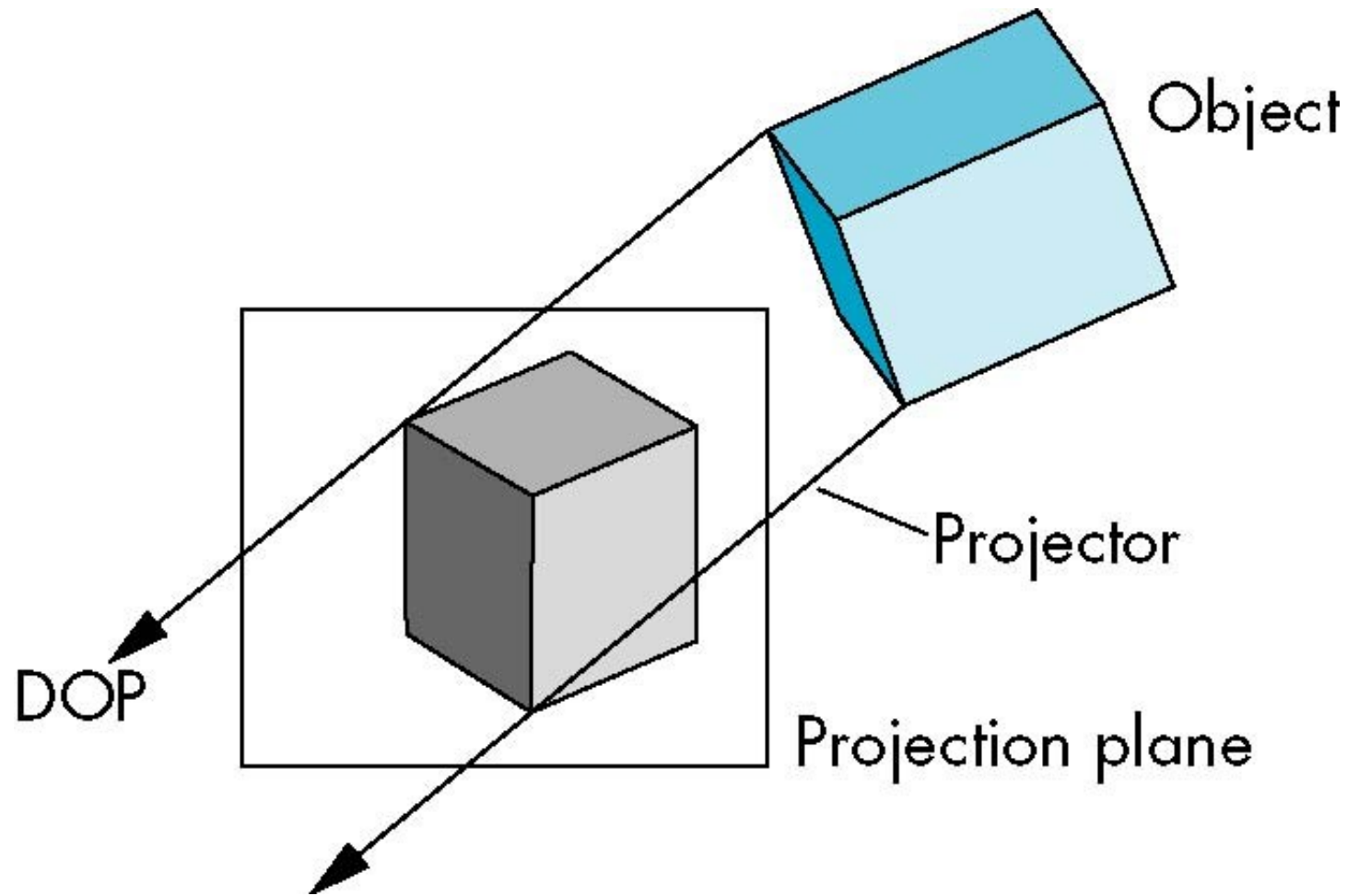
Orthogonal Normalization

`ortho(left, right, bottom, top, near, far)`

normalization $\Rightarrow$ find transformation to convert specified clipping volume to default



Parallel Projection



Orthogonal Matrix

- Two steps

- Move center to origin

$$T(-(left+right)/2, -(bottom+top)/2, (near+far)/2))$$

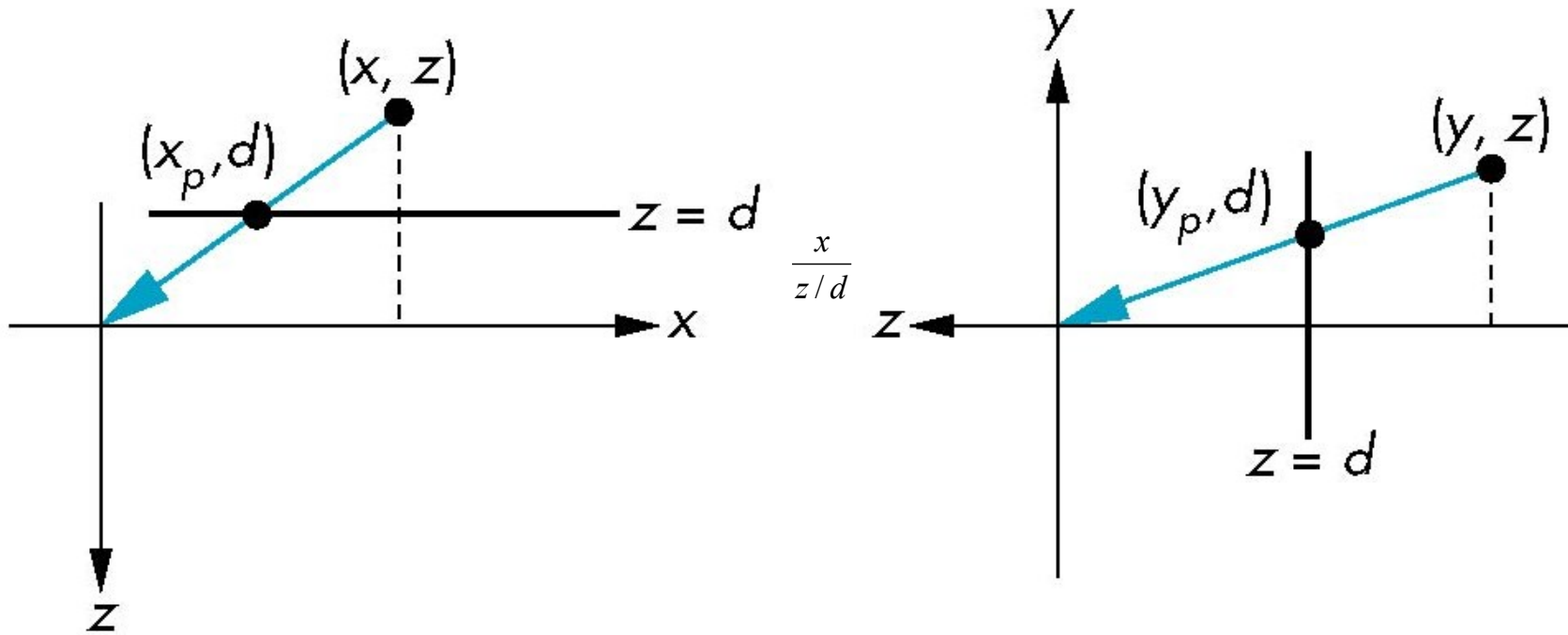
- Scale to have sides of length 2

$$S(2/(left-right), 2/(top-bottom), 2/(near-far))$$

$$\mathbf{P} = \mathbf{ST} = \begin{bmatrix} \frac{2}{right-left} & 0 & 0 & -\frac{right-left}{right-left} \\ 0 & \frac{2}{top-bottom} & 0 & -\frac{top-bottom}{top-bottom} \\ 0 & 0 & \frac{2}{near-far} & \frac{far-near}{far-near} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Perspective Equations

Consider top and side views

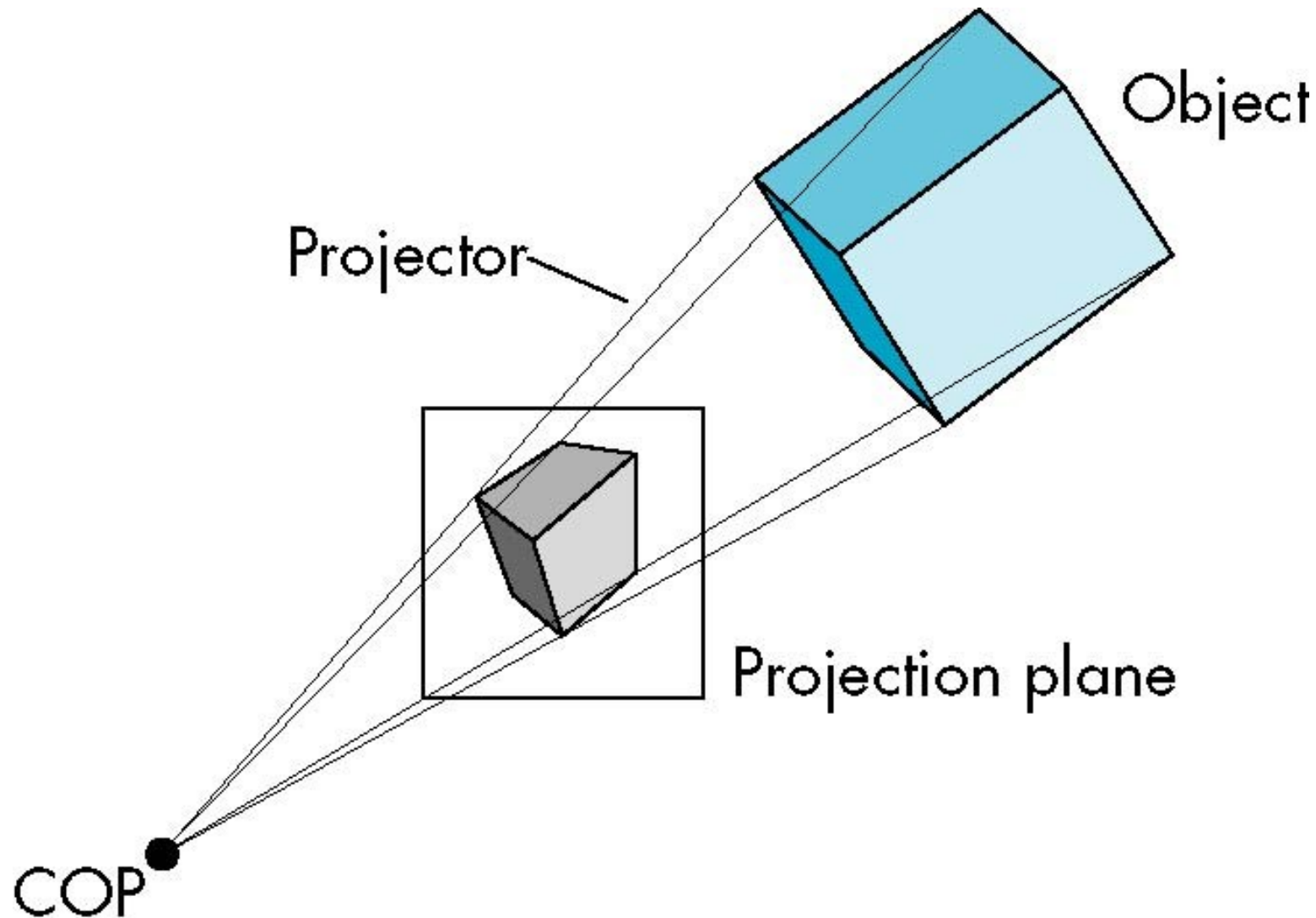


$$x_p = \frac{x}{z/d}$$

$$y_p = \frac{y}{z/d}$$

$$z_p = d$$

Perspective Projection



Homogeneous Coordinate Form

consider $\mathbf{q} = \mathbf{M}\mathbf{p}$ where $\mathbf{M} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1/d & 0 \end{bmatrix}$

$$\mathbf{q} = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \Rightarrow \mathbf{p} = \begin{bmatrix} x \\ y \\ z \\ z/d \end{bmatrix}$$

Homogeneous Coordinate Form

$$\mathbf{q} = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \Rightarrow \mathbf{p} = \begin{bmatrix} x \\ y \\ z \\ z/d \end{bmatrix}$$

Perspective Division

- However $w \neq 1$, so we must divide by w to return from homogeneous coordinates
- This *perspective division* yields

$$x_p = \frac{x}{z/d} \quad y_p = \frac{y}{z/d} \quad z_p = d$$

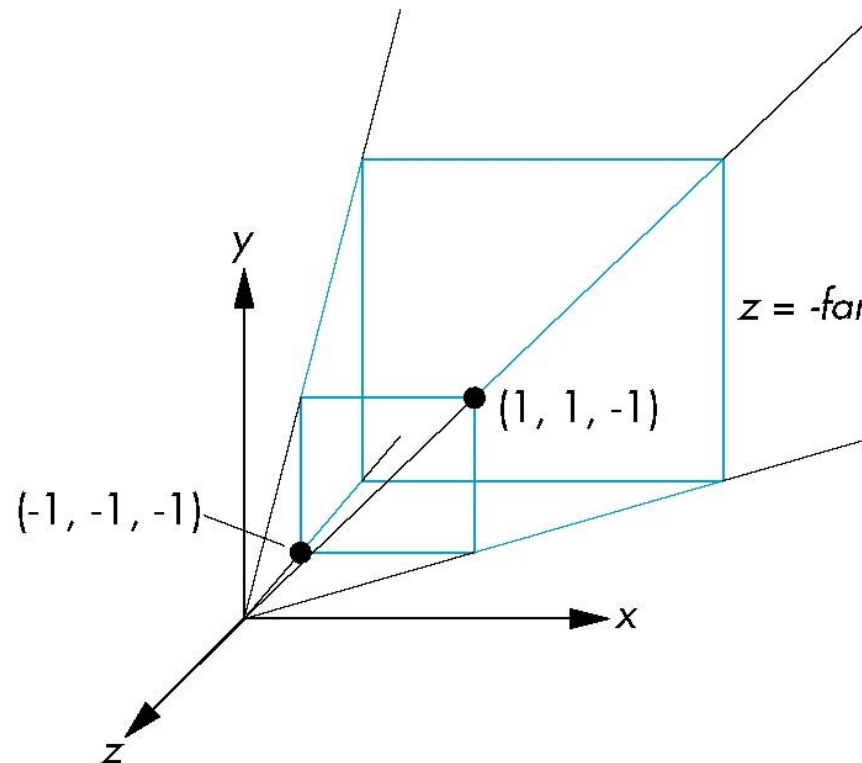
the desired perspective equations

- We will consider the corresponding clipping volume with MV.js functions that are equivalent to deprecated OpenGL functions

Simple Perspective

Consider a simple perspective with the COP at the origin, the near clipping plane at $z = -1$, and a 90 degree field of view determined by the planes $x = \pm z$, $y = \pm z$

$$x = \pm z, y = \pm z$$



Perspective Matrices

Simple projection matrix in homogeneous coordinates

$$\mathbf{M} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Note that this matrix is independent of the far clipping plane

Generalization

$$\mathbf{N} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \alpha & \beta \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

after perspective division, the point $(x, y, z, 1)$ goes to

$$x'' = x/z$$

$$y'' = y/z$$

$$Z'' = -(\alpha + \beta/z)$$

which projects orthogonally to the desired point regardless of α and β

Picking α and β

If we pick

$$\alpha = \frac{\text{near} + \text{far}}{\text{far} - \text{near}}$$

$$\beta = \frac{2\text{near} * \text{far}}{\text{near} - \text{far}}$$

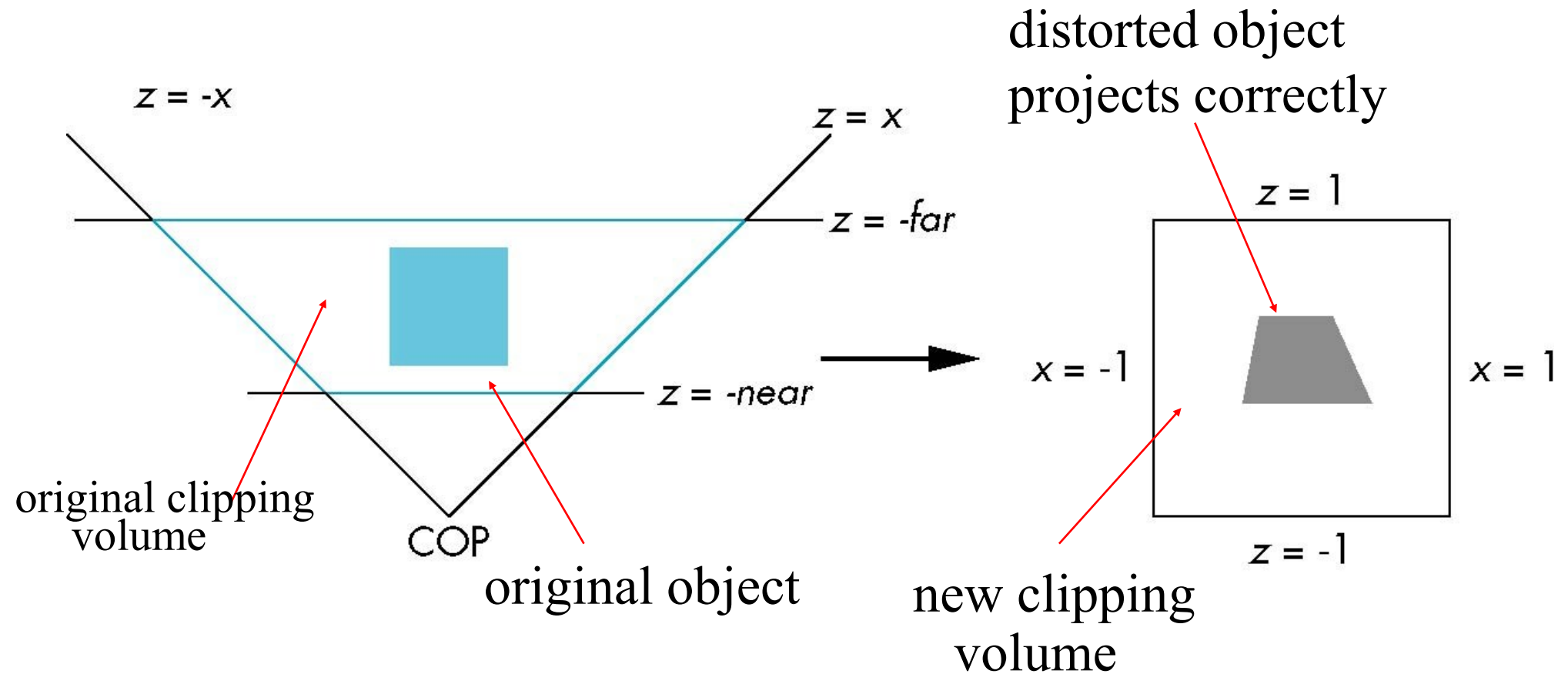
the near plane is mapped to $z = -1$

the far plane is mapped to $z = 1$

and the sides are mapped to $x = \pm 1, y = \pm 1$

Hence the new clipping volume is the default clipping volume

Normalization Transformation



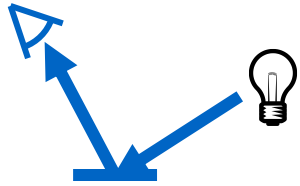
Normalization and Hidden-Surface Removal

- Although our selection of the form of the perspective matrices may appear somewhat arbitrary, it was chosen so that if $z_1 > z_2$ in the original clipping volume then the for the transformed points $z_1' > z_2'$
- Thus hidden surface removal works if we first apply the normalization transformation
- However, the formula $z'' = -(\alpha + \beta/z)$ implies that the distances are distorted by the normalization which can cause numerical problems especially if the near distance is small

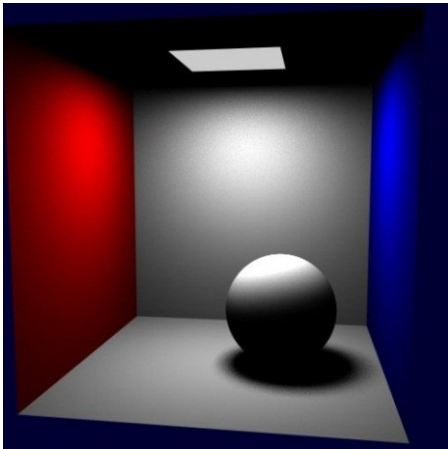
Illumination

$$K_d I_d (\mathbf{n} \cdot \mathbf{L}) + K_s I_s (\mathbf{v} \cdot \mathbf{r})^\alpha + K_a I_a$$

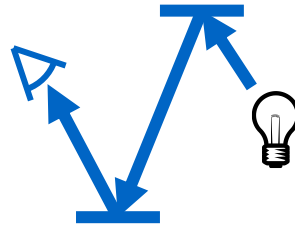
Local vs Global Rendering



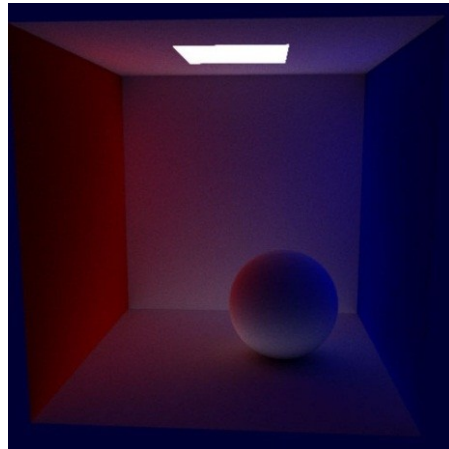
Direct illumination



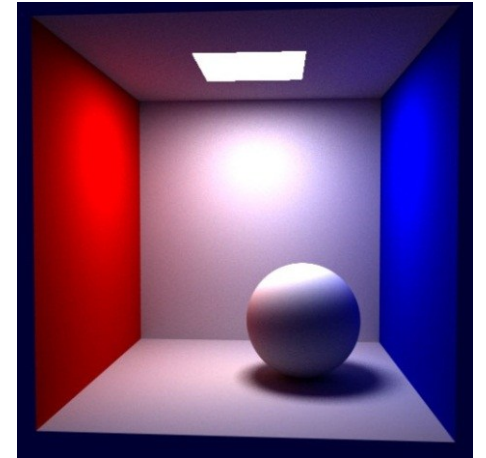
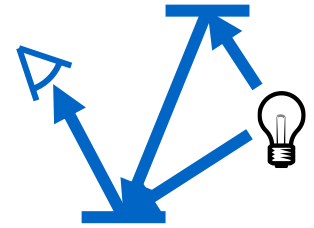
+



Indirect illumination

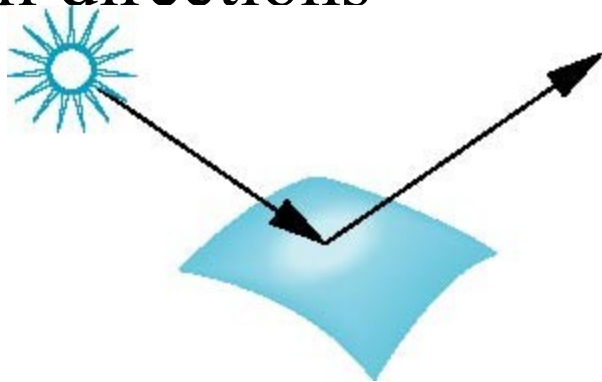


=

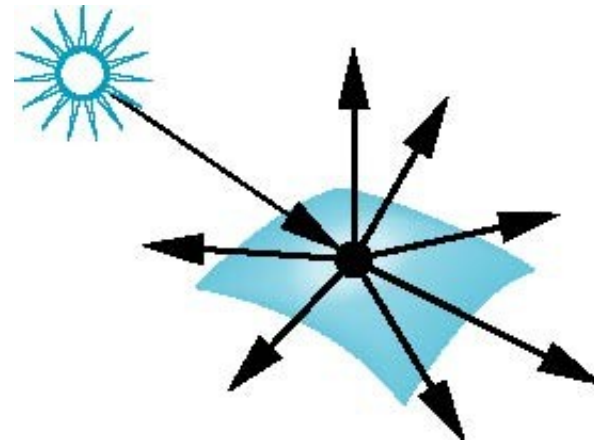


Surface Types (Direct-Interaction)

- The smoother a surface, the more reflected light is concentrated in the direction a perfect mirror would reflect the light
- A very rough surface scatters light in all directions



smooth surface



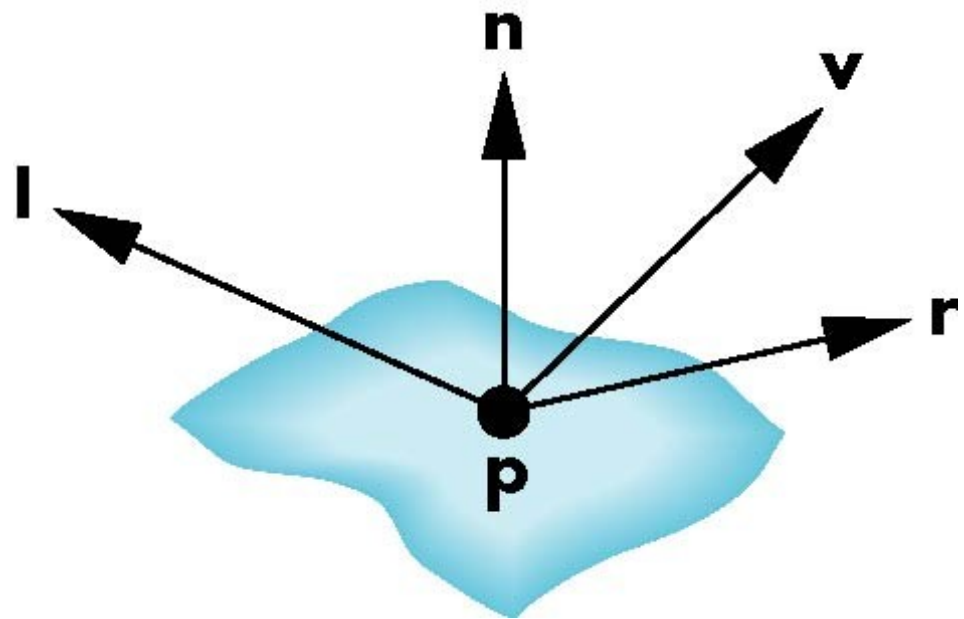
rough surface

Light-Material Interaction

- Light that strikes an object is partially absorbed and partially scattered (reflected)
- The amount reflected determines the color and brightness of the object
 - A surface appears red under white light because the red component of the light is reflected and the rest is absorbed
- The reflected light is scattered in a manner that depends on the smoothness and orientation of the surface

Phong Model

- A simple model that can be computed rapidly
- Has three components
 - Diffuse
 - Specular
 - Ambient
- Uses four vectors
 - To source
 - To viewer
 - Normal
 - Perfect reflector

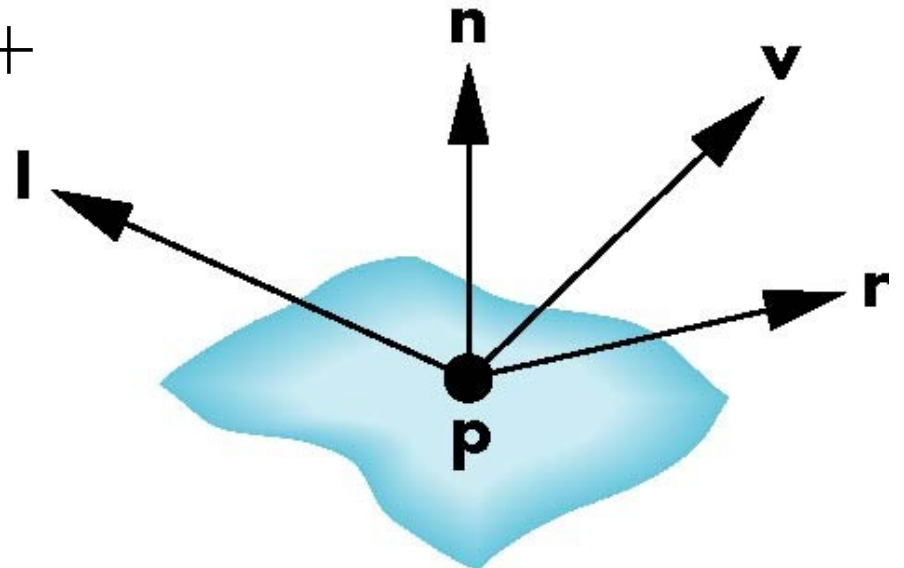


Phong Model

For each light source and each color component, the Phong model can be written (without the distance terms) as

$$I = k_d I_d \mathbf{l} \cdot \mathbf{n} + k_s I_s (\mathbf{v} \cdot \mathbf{r})^\alpha + k_a I_a$$

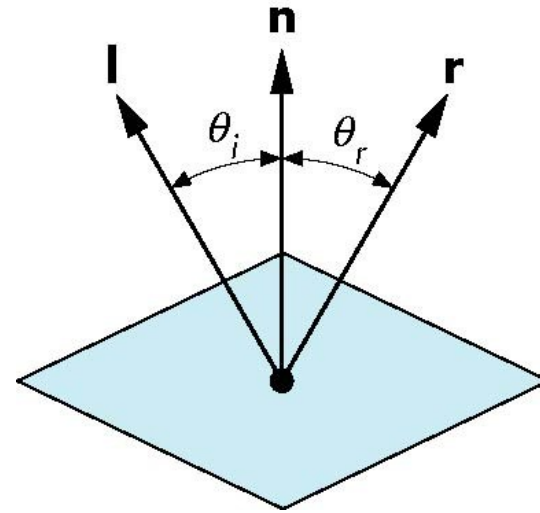
For each color component we add contributions from all sources



Ideal Reflector

- Normal is determined by local orientation
- Angle of incidence = angle of reflection
- The three vectors must be coplanar

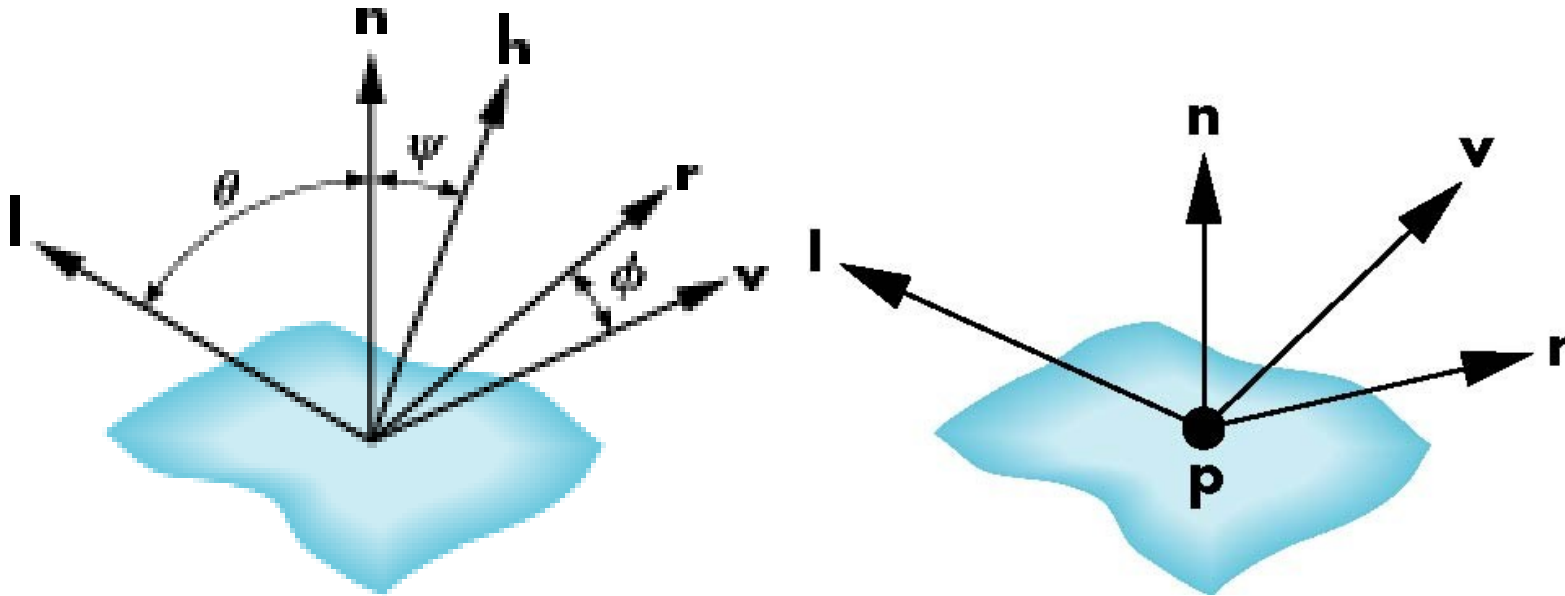
$$\mathbf{r} = 2 (\mathbf{l} \cdot \mathbf{n}) \mathbf{n} - \mathbf{l}$$



Blinn-Phong Model

9
3

$$\mathbf{h} = (\mathbf{l} + \mathbf{v}) / |\mathbf{l} + \mathbf{v}|$$



Blinn-Phong Model

For each light source and each color component, the Blinn-Phong model can be written (without the distance terms) as

$$I = k_d I_d \mathbf{l} \cdot \mathbf{n} + k_s I_s (\mathbf{n} \cdot \mathbf{h})^\beta + k_a I_a$$

For each color component we add contributions from all sources

Light Sources

- In the Phong Model, we add the results from each light source
- Each light source has separate diffuse, specular, and ambient terms to allow for maximum flexibility even though this form does not have a physical justification
- Separate red, green and blue components
- Hence, 9 coefficients for each point source
 - $I_{dr}, I_{dg}, I_{db}, I_{sr}, I_{sg}, I_{sb}, I_{ar}, I_{ag}, I_{ab}$

Material Properties

- Material properties match light source properties
 - Nine absorption coefficients
 - k_{dr} , k_{dg} , k_{db} , k_{sr} , k_{sg} , k_{sb} , k_{ar} , k_{ag} ,
 k_{ab}
 - Shininess coefficient α

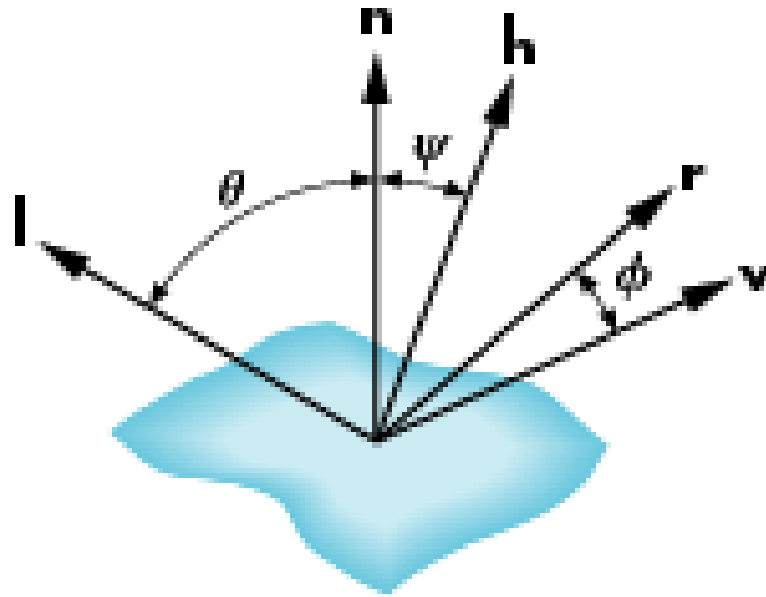
Modified Phong Model

- The specular term in the Phong model is problematic because it requires the calculation of a new reflection vector and view vector for each vertex
- Blinn suggested an approximation using the halfway vector that is more efficient

The Halfway Vector

- **h** is normalized vector halfway between **l** and **v**

$$\mathbf{h} = (\mathbf{l} + \mathbf{v}) / |\mathbf{l} + \mathbf{v}|$$



Using the halfway vector

- Replace $(\mathbf{v} \cdot \mathbf{r})^\alpha$ by $(\mathbf{n} \cdot \mathbf{h})^\beta$
- β is chosen to match shininess
- Note that halfway angle is half of angle between $\mathbf{r}$ and $\mathbf{v}$ if vectors are coplanar
- Resulting model is known as the modified Phong or Phong-Blinn lighting model
 - Specified in OpenGL standard

Computation of Vectors

- $\mathbf{l}$ and $\mathbf{v}$ are specified by the application
- Can compute $\mathbf{r}$ from $\mathbf{l}$ and $\mathbf{n}$
- Problem is determining $\mathbf{n}$
- For simple surfaces $\mathbf{n}$ can be determined but how we determine $\mathbf{n}$ differs depending on underlying representation of surface
- OpenGL leaves determination of normal to application
 - Exception for GLU quadrics and Bezier surfaces was deprecated

WebGL lighting

- Need
 - Normals
 - Material properties
 - Lights
- State-based shading functions have been deprecated (`glNormal`, `glMaterial`, `glLight`)
- Compute in application or in shaders

Normalization

- Cosine terms in lighting calculations can be computed using dot product
- Unit length vectors simplify calculation
- Usually we want to set the magnitudes to have unit length but
 - Length can be affected by transformations
 - Note that scaling does not preserve length
- GLSL has a normalization function

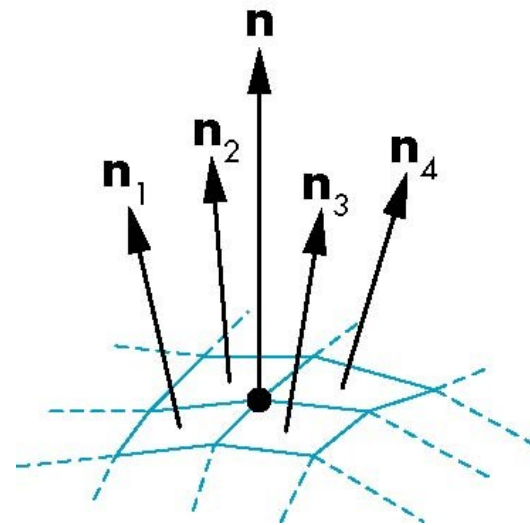
Polygonal Shading

- In per vertex shading, shading calculations are done for each vertex
 - Vertex colors become vertex shades and can be sent to the vertex shader as a vertex attribute
 - Alternately, we can send the parameters to the vertex shader and have it compute the shade
- By default, vertex shades are interpolated across an object if passed to the fragment shader as a varying variable (smooth shading)
- We can also use uniform variables to shade with a single shade (flat shading)

Mesh Shading

- The previous example is not general because we knew the normal at each vertex analytically
- For polygonal models, Gouraud proposed we use the average of the normals around a mesh vertex

$$\mathbf{n} = (\mathbf{n}_1 + \mathbf{n}_2 + \mathbf{n}_3 + \mathbf{n}_4) / |\mathbf{n}_1 + \mathbf{n}_2 + \mathbf{n}_3 + \mathbf{n}_4|$$



Gouraud and Phong Shading

- Gouraud Shading
 - Find average normal at each vertex (vertex normals)
 - Apply modified Phong model at each vertex
 - Interpolate vertex shades across each polygon
- Phong shading
 - Find vertex normals
 - Interpolate vertex normals across edges
 - Interpolate edge normals across polygon
 - Apply modified Phong model at each fragment

Comparison

- If the polygon mesh approximates surfaces with a high curvatures, Phong shading may look smooth while Gouraud shading may show edges
- Phong shading requires much more work than Gouraud shading
 - Until recently not available in real time systems
 - Now can be done using fragment shaders
- Both need data structures to represent meshes so we can obtain vertex normals

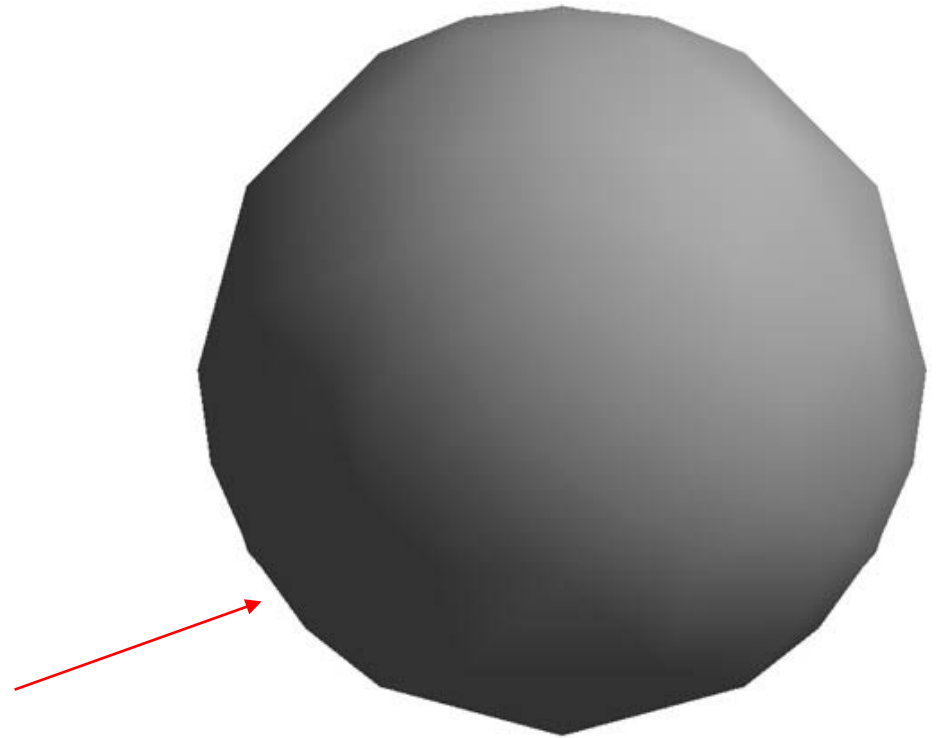
Polygon Normals

- Triangles have a single normal
 - Shades at the vertices as computed by the modified Phong model can be almost same
 - Identical for a distant viewer (default) or if there is no specular component
- Consider model of sphere
- Want different normals at each vertex even though this concept is not quite correct mathematically



Smooth Shading

- We can set a new normal at each vertex
- Easy for sphere model
 - If centered at origin $\mathbf{n} = \mathbf{p}$
- Now smooth shading works
- Note *silhouette edge*

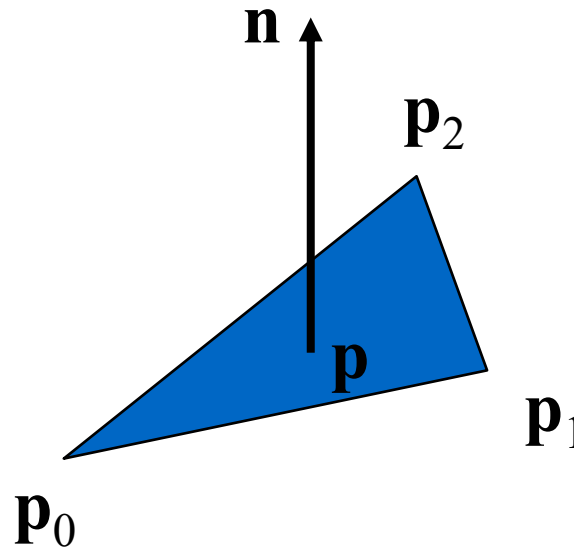


Normal for Triangle

plane $\mathbf{n} \cdot (\mathbf{p} - \mathbf{p}_0) = 0$

$$\mathbf{n} = (\mathbf{p}_2 - \mathbf{p}_0) \times (\mathbf{p}_1 - \mathbf{p}_0)$$

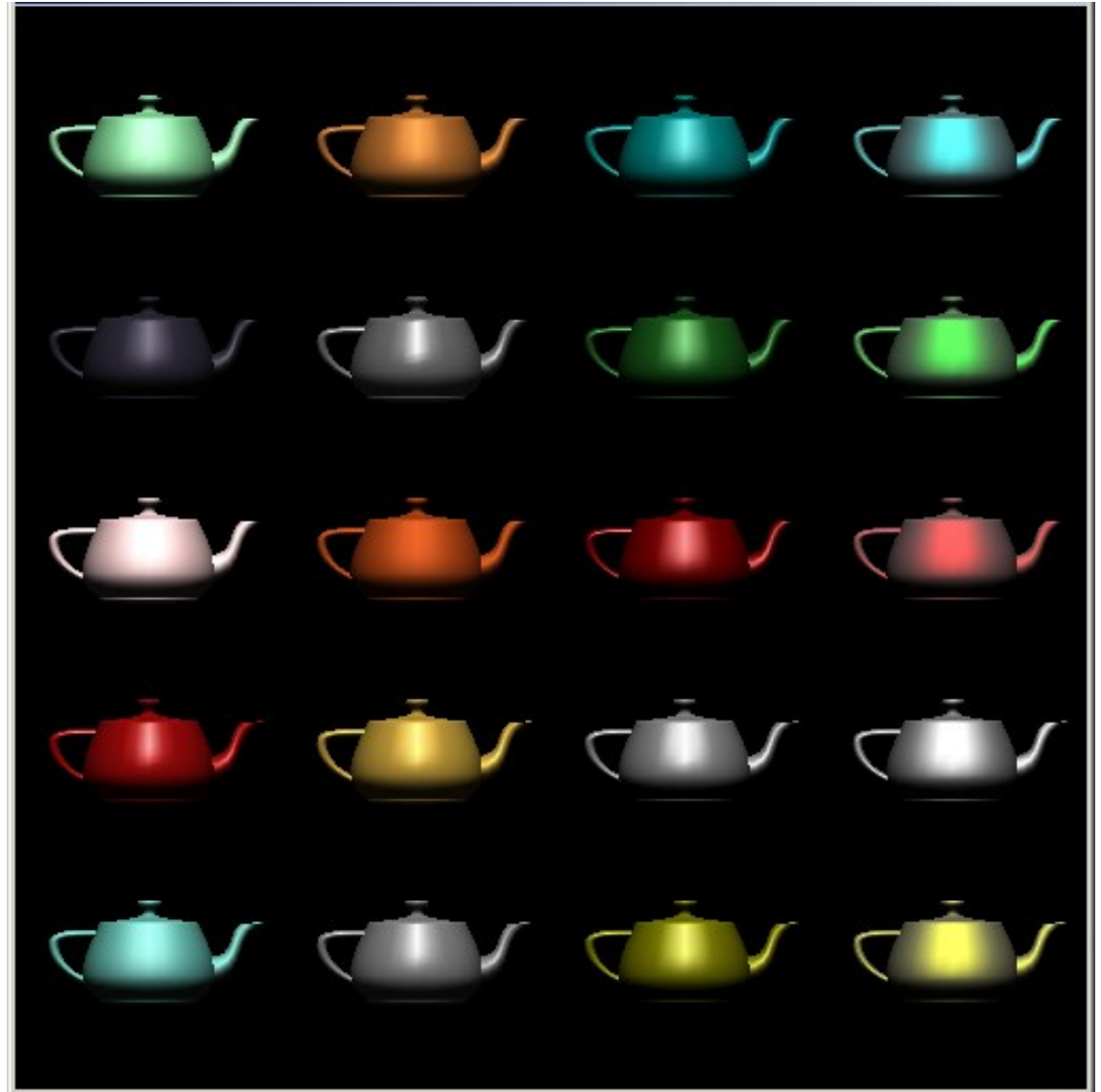
normalize $\mathbf{n} \leftarrow \mathbf{n} / |\mathbf{n}|$



Note that right-hand rule determines outward face

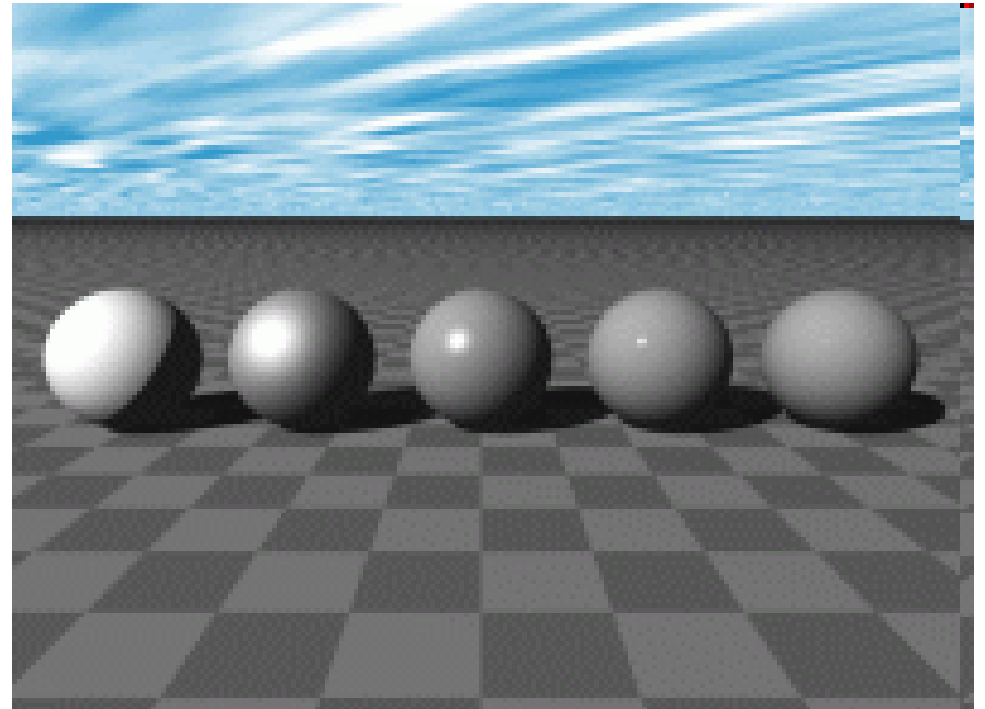
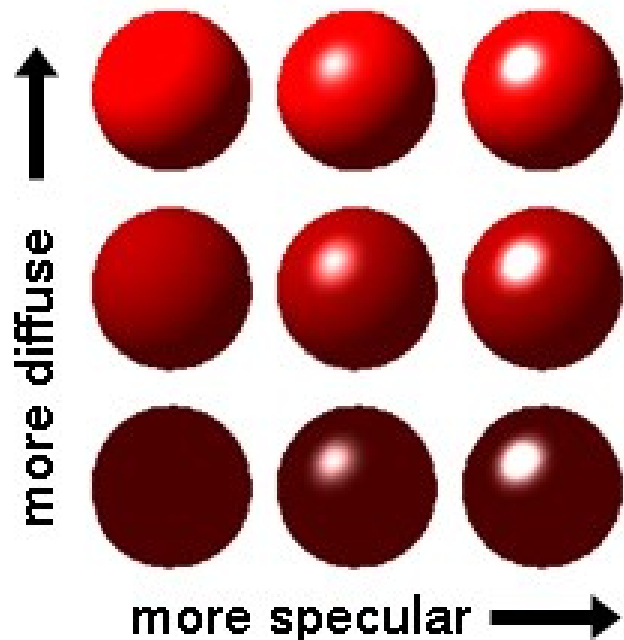
Example

Only differences in these teapots are the parameters in the modified Phong model



Modeling Specular Reflections

- Phong proposed using a term that dropped off as the angle between the viewer and the ideal reflection increased



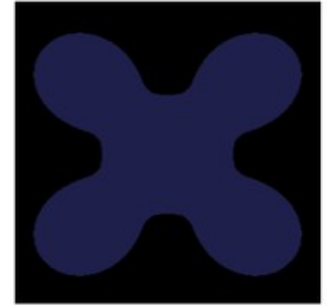
Ambient Light

- Ambient light is the result of multiple interactions between (large) light sources and the objects in the environment
- Amount and color depend on both the color of the light(s) and the material properties of the object
- Add $k_a I_a$ to diffuse and specular terms

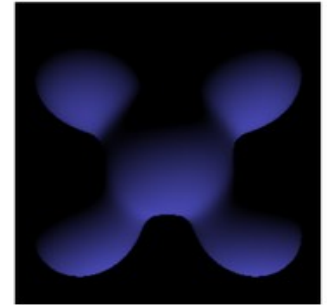
reflection coef

intensity of ambient light

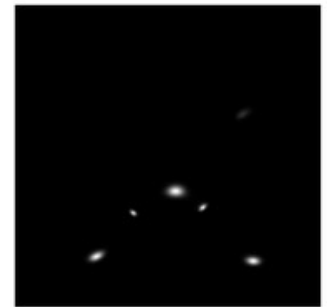
Ambient



Diffuse



Specular



Phong

